SOFTOMATIC
TECHNOLOGY SERVICES

EXAMATE · SYSTEM DESIGN · ELITE + BATCH 1

Lesson 7 · Scaling Databases: Replication and Sharding

The two moves every growing database makes: copying it and splitting it, and the new bugs each move brings. Told through one story: a Diwali-sale link on our short-link service sho.rt that was saved correctly and still said "not found".

MODULE
System Design · Part A
Foundations

TIME
About 80 minutes
plus 45 minutes of practice problems

YOU NEED
Lessons 4, 5 and 6
arithmetic, the hash ring, caching

YOUR ROUTE TODAY

01 Riya's link says "not found"
02 Copy it: replicas and lag
03 Sync, semi-sync, multi-leader
04 Split it: shard keys, resharding
05 **Rings, fan-out, hot keys**

LESSON 7 AT A GLANCE

What you will be able to do

Replicas and lag

How leader-follower works, why replicas lag, and how to measure it on PostgreSQL or MySQL.

Three lag bugs

Missing your own write, time going backwards, lost writes on failover, each with a fix.

Async vs sync

The cost of each mode in milliseconds and in lost writes.

Multi-leader

When it is worth it, and how to avoid or resolve write conflicts.

Shard keys

Range, hash or directory, and defending the key against the hot customer.

Resharding and fan-out

Data moved under mod-N, rings and logical shards; cross-shard queries and their cost.

Subject / module	System Design / Part A: Foundations
Difficulty	Intermediate. You need Lesson 4's arithmetic, Lesson 5's hash ring, and Lesson 6's negative caching.
Prerequisites	Lessons 1 to 6. Lesson 4 sized sho.rt at 7,716 redirects per second at peak, about 77 link creations per second at peak, and 6 billion links (1.2 to 3 TB) in five years. Lesson 5 introduced consistent hashing as a routing tool for app servers. Lesson 6 put Redis in front of the database, cut database reads to about 772 per second at a 90% hit rate, and cached "not found" for 60 seconds.
Next lesson	Lesson 8: Choosing a datastore: relational, key-value, document, wide-column, graph and search
Companies covered	PhonePe and Atlassian (dated reports). Every other question is Pattern-only.
Commands and configs	PostgreSQL pg_stat_replication lag columns and synchronous_standby_names / synchronous_commit; MySQL SHOW REPLICA STATUS and semisynchronous replication; a 30-line Python hash-ring simulation you can run.
Checklist	☐ read replicas and replication lag ☐ leader-follower vs multi-leader ☐ sharding strategies (range, hash, directory) ☐ resharding ☐ consistent hashing with virtual nodes ☐ cross-shard queries

HOW TO READ THIS LESSON

Outcomes and evidence labels

What you will be able to do after this lesson:

- Explain how leader-follower replication works, why replicas lag, and measure the lag on a real PostgreSQL or MySQL server.
- Name the bugs that lag causes (a reader missing their own write, time going backwards between two reads, data lost on failover) and fix each one with a specific technique.
- Compare asynchronous, semisynchronous and synchronous replication with the cost of each in milliseconds and in lost writes.
- Say when multi-leader replication is worth it, and how to avoid or resolve write conflicts.
- Choose between range, hash and directory sharding for a given access pattern, and defend the shard key against "what about the hot customer?"
- Calculate how much data moves when you add a shard under mod-N, consistent hashing, and fixed logical shards, and plan an online resharding.
- Handle cross-shard queries (scatter-gather, top-k, joins, unique constraints, transactions) and explain their cost.

SOURCE-CONFIRMED

A dated first-person report naming the company and the question or topic, October 2024 to October 2026, opened in full.

CANDIDATE-REPORTED

Compilations, undated posts, older reports.

PATTERN-ONLY

Commonly asked, no company source, so no company name. Practice problems and quiz questions are original and unlabelled.

AN HONEST NOTE ON THE EVIDENCE

Two dated reports name this lesson's topics. A **PhonePe** SDE 2 candidate (August 2025) was asked by the hiring manager what the replication and sharding strategy for their project's databases was, and how much load it handled [S7]. An **Atlassian** SDE 2 candidate (August 2025) was asked in the screening round what challenges arise when consistent hashing is used to distribute songs across servers [S8]. Interview-prep compilations list sharding, consistent hashing and replication among the most frequent deep-dive topics [S9], but compilations are Candidate-reported. No dated sharding question was found for Amazon, Flipkart or Uber in the window, so those names do not appear below.

INTERVIEW MODE: SAY IT LIKE THIS

- This lesson covers read replicas and replication lag, leader-follower and multi-leader replication, range, hash and directory sharding, resharding, consistent hashing with virtual nodes, and cross-shard queries.

9:00 PM, THE NIGHT OF A DIWALI SALE

Saved correctly. Still "not found".

Riya runs a small online saree store. She creates a short link for her Diwali sale on sho.rt and pastes it into her WhatsApp broadcast list of **4,000 customers**. The first customer taps it 0.3 seconds later.

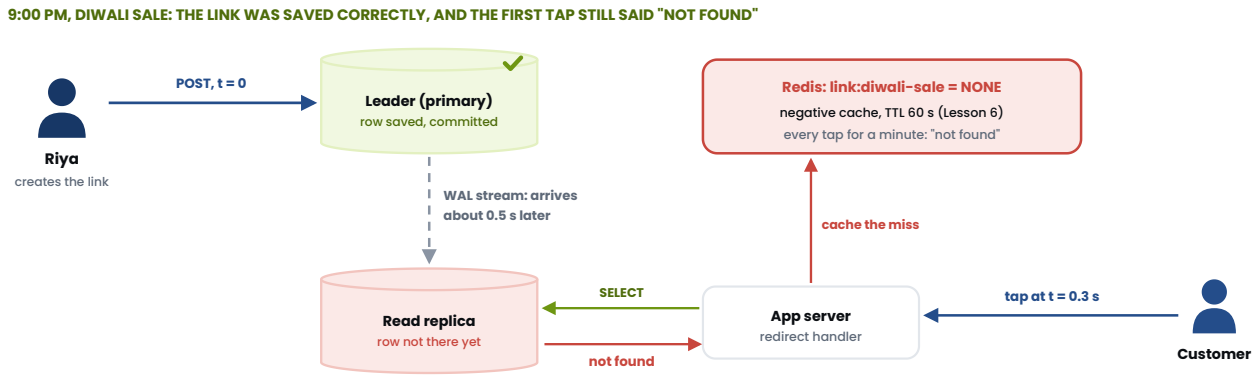
They get **"Link not found"**. They tap again. Still not found. They tap a third time a minute later, and it works. In that first minute, **600 customers** saw "not found" and **about 200** of them never came back.

Nothing crashed. The link was saved correctly. The database did exactly what it was told.

PAUSE AND PREDICT

What went wrong, using only ideas from Lessons 4 to 6 plus one new word, **replica**?

Answer: The link was written to the **primary** database. Redirects read from a **read replica**, which receives changes a fraction of a second later. The first tap reached the replica before the new row did, so the replica said "no such code". Lesson 6's **negative cache** then stored NONE for that code for 60 seconds, so every tap in that minute got "not found" from Redis without even asking the database.



First minute: 600 customers saw "Link not found"; about 200 never came back. Nothing crashed. The bug lives in the gap between two healthy nodes.

This lesson is about the two moves that every growing database makes, **copying it** (replication) and **splitting it** (sharding), and about the new bugs each move brings.

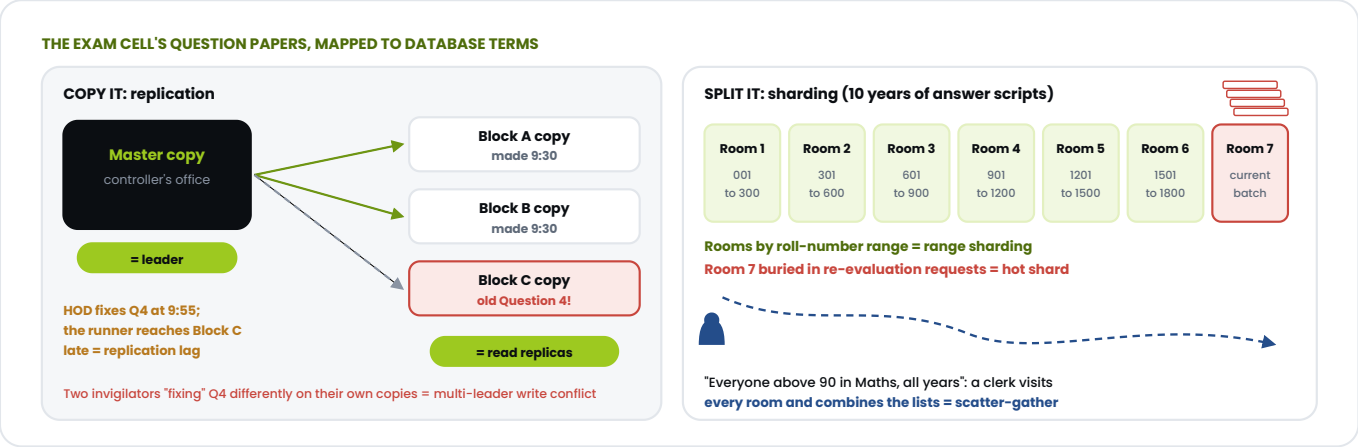
INTERVIEW MODE: SAY IT LIKE THIS

- Read replicas apply the primary's changes after a delay, so a read sent to a replica right after a write can miss that write.
- If a negative cache then stores the miss, the error lasts as long as the cache TTL. Replication and sharding both scale a database, and both introduce failure modes that the design must handle.

BEFORE THE JARGON

The exam cell's question papers

Your college exam cell has one master copy of every question paper, locked in the controller's office.



- **Replication.** On exam day, 30 classrooms need the paper. Nobody walks 1,200 students to the controller's office. The office makes photocopies and sends them to each block. Reading is now fast and local. That is a **read replica**.
- **Replication lag.** At 9:55 the HOD finds a typo in Question 4 and corrects the master. The photocopies in Block C were made at 9:30. Block C's students read the old Question 4 until a runner brings the correction. The delay between the master changing and a copy changing is **replication lag**.
- **Why only one master.** If Block A's invigilator and Block C's invigilator both "fix" Question 4 on their own copies, differently, which version is correct? Having one place where changes are made (**leader-follower**) avoids that argument. Allowing several places to make changes (**multi-leader**) is faster for each block but creates conflicts someone must resolve.
- **Sharding.** The exam cell also stores 10 years of answer scripts. One room cannot hold them. So scripts for roll numbers 001 to 300 go to Room 1, 301 to 600 to Room 2, and so on. Each room holds part of the data. That is a **shard**, and "roll-number ranges" is a **range-based shard key**.
- **The hot room.** Re-evaluation requests always come for the current batch, all of which sits in Room 7. Room 7's clerk is buried; Rooms 1 to 6 are idle. That is a **hot shard**.
- **Cross-shard queries.** "List every student who scored above 90 in Maths, across all years" needs a clerk to visit every room and combine the lists. That is a **scatter-gather** query.

THE WHOLE LESSON IN ONE TABLE

Exam cell to database term

EXAM CELL	DATABASE TERM
Master copy in the controller's office	Leader (primary)
Photocopies in each block	Followers (read replicas)
Correction reaching Block C late	Replication lag
Two invigilators "fixing" the same question differently	Multi-leader write conflict
Scripts split into rooms by roll number	Range sharding
Room 7 buried in re-evaluation requests	Hot shard
A clerk visiting every room for one report	Scatter-gather (cross-shard query)

WHAT GOES WRONG

Where the analogy breaks. Photocopies are made in batches; database replicas receive a **continuous stream** of changes, usually well under a second behind [S1]. And rooms are filled by a human's rule; in a real system you choose the rule (the shard key) once, and changing it later means moving terabytes, which is why this lesson spends so long on choosing it.

Just for fun. Every college has the one invigilator who announces the corrected question only to the first row. That invigilator is an asynchronous replica with a bad network.

TINY CHALLENGE

In the analogy, what would "synchronous replication" look like? Answer: The HOD does not let the exam start until every block confirms it has the corrected paper. Safer, but the whole exam waits for the slowest block.

MEMORY HOOK: COPY VS SPLIT

Copy the paper so more people can **read**. **Split** the scripts so no room has to **hold** everything.

INTERVIEW MODE: SAY IT LIKE THIS

- Replication keeps full copies of the data on several machines to scale reads and survive failures; sharding splits the data so each machine holds only part of it, to scale writes and storage.
- One leader accepting writes avoids conflicts; several leaders reduce write latency but need conflict resolution.

EVERY WORD, ONE LINE (1 OF 2)

Vocabulary: replication

Every term is defined before it is used. Terms from earlier lessons (QPS, peak, p99, TTL, cache hit, negative cache, hash ring) are reused without re-definition.

Replication
Keeping a copy of the same data on more than one machine.

Follower (replica, standby)
A node that receives the leader's changes and serves reads. Older docs say "slave".

WAL (write-ahead log)
PostgreSQL's log of every change, written before the data files. Replicas replay it [S1].

Replication lag
The delay between a commit on the leader and that change becoming visible on a follower.

Synchronous replication
The leader waits for one or more followers to confirm before replying to the client [S1].

Failover
Promoting a follower to leader when the leader dies.

Monotonic reads
A guarantee that a user never sees data go backwards in time across two reads.

Write conflict
Two leaders accept different changes to the same item before hearing about each other.

Leader (primary, source)
The node that accepts writes. Older docs say "master".

Read replica
A follower used to take read traffic off the leader.

Binary log (binlog)
MySQL's equivalent log of changes that replicas fetch and apply [S2].

Asynchronous replication
The leader commits and replies without waiting for any follower. Default in PostgreSQL and MySQL [S1, S2].

Semisynchronous replication
MySQL mode: the leader waits until at least one replica has received and logged the change, not applied it [S2].

Read-your-writes
A guarantee that a user always sees their own recent writes.

Multi-leader replication
More than one node accepts writes and they replicate to each other [S10].

Last write wins (LWW)
Conflict rule: keep the change with the latest timestamp, discard the other [S3].

EVERY WORD, ONE LINE (2 OF 2)

Vocabulary: sharding

Partitioning
Splitting a table into parts; often used for splits inside one database server [S6].

Shard key (partition key)
The column whose value decides which shard a row goes to [S4, S5].

Hash sharding
Each row goes to the shard chosen by a hash of its shard key.

Logical shard
A unit of data (for example, a Postgres schema) that can move between machines as a whole [S4, S5].

Hot shard / hot key
A shard or key that receives far more traffic than average.

Consistent hashing
Placing nodes and keys on a hash ring so that adding a node moves only about 1/N of the keys (Lesson 5) [S3].

Scatter-gather
Sending one query to many shards and merging the results.

Two-phase commit (2PC)
A protocol for one transaction across several databases: prepare everywhere, then commit everywhere [S6].

Sharding
Splitting data across multiple machines; each machine holds a shard [S6].

Range sharding
Each shard owns a continuous range of shard-key values.

Directory sharding
A lookup table records which shard each key (or group of keys) lives on.

Physical shard
An actual database server holding one or more logical shards.

Resharding (rebalancing)
Moving data between shards when you add, remove or split them.

Virtual node (vnode, token)
One of many ring positions owned by the same physical node [S3].

Fan-out
The number of shards one request touches.

CAP (named only)
A theorem about what a distributed system can guarantee during a network partition. Lesson 9.

INTERVIEW MODE: SAY IT LIKE THIS

- A leader accepts writes and followers replay its log. Replication lag is the delay before a follower shows a committed change.
- A shard key decides which machine a row lives on, and consistent hashing with virtual nodes limits how much data moves when machines are added.

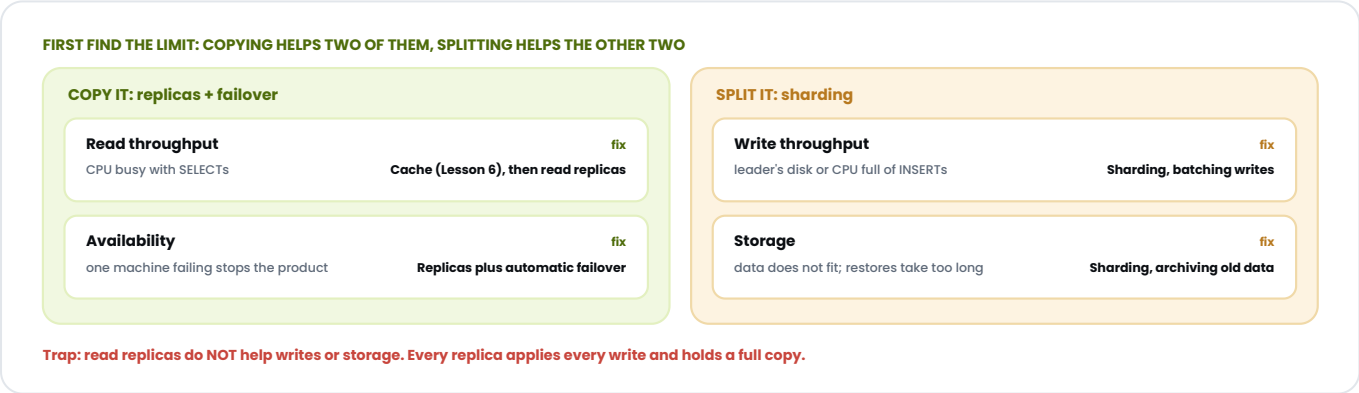
CONCEPT 1 · WHY ONE DATABASE STOPS BEING ENOUGH

Copy or split? First find the limit

Nine steps, bottom-up. The running example is sho.rt. Each concept has the problem it solves, how it works, a worked example with numbers, and what goes wrong.

The problem it solves. You need to know which limit you are hitting before you pick a fix. Replication and sharding solve different problems. A single database server runs out of one of four things:

LIMIT	SYMPTOM	FIX THAT HELPS	FIX THAT DOES NOT HELP
Read throughput	CPU busy with SELECTs, read latency climbing	Cache (Lesson 6), then read replicas	Sharding alone (overkill)
Availability	One machine failing takes the product down	Replicas plus automatic failover	Bigger machine
Write throughput	Leader's disk or CPU saturated by INSERT/UPDATE	Sharding, batching writes	Read replicas (every replica still applies every write)
Storage	Data no longer fits, or backups and restores take too long	Sharding, archiving old data	Read replicas (each holds a full copy)



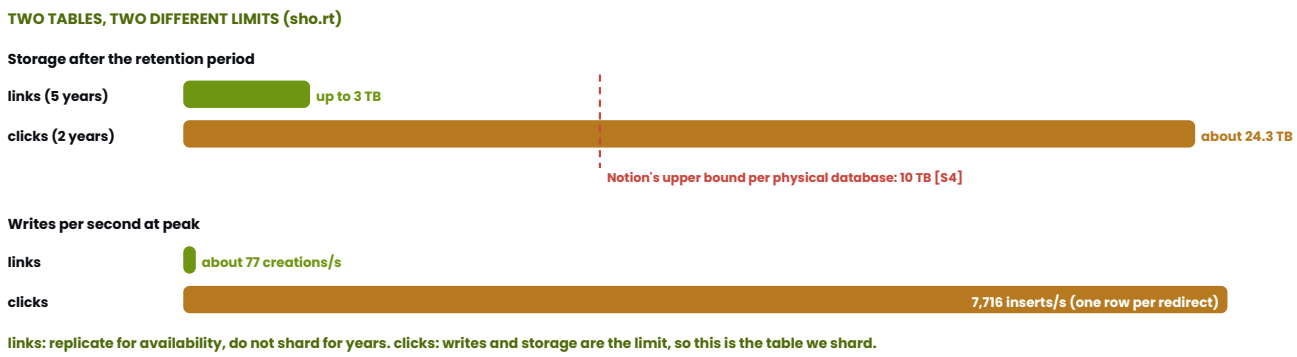
Worked example (the sho.rt links table). From Lessons 4 and 6: writes about **77 creations per second** at peak; reads reaching the database about **772 per second** at a 90% cache hit rate; storage **1.2 to 3 TB** after five years. All three fit on one well-sized PostgreSQL server (assumption: a single node handles thousands of simple queries per second and several terabytes of disk). Notion, for comparison, set an upper bound of **10 TB per physical database** when it planned its fleet [S4]. So the links table needs replicas for availability, but it does not need sharding for years.

CONCEPT 1 · A NEW TABLE: CLICK EVENTS

The table that does need splitting

The product team now wants per-link analytics: every redirect logs one row (code, timestamp, country, device, referrer). Lesson 4's average of **3,860 redirects per second** gives:

QUANTITY	CALCULATION	RESULT
Rows per day	3,860 x 86,400	about 333.5 million
Bytes per row (assumption: about 35 bytes of data, padded to 100 with row and index overhead)		100 B
Data per day	333.5 million x 100 B	about 33 GB
Data per year	33.35 GB x 365	about 12.2 TB
Two years kept	12.2 x 2	about 24.3 TB
Peak insert rate	Lesson 4 peak	about 7,716 per second



24 TB and 7,716 inserts per second on one leader is where trouble starts: backups and restores take many hours, and every replica must also absorb every insert. **The click table is the one we will shard.**

WHAT GOES WRONG

Sharding everything on day one. You pay the complexity (routing, cross-shard queries, resharding) years before the data needs it.

Just for fun. Sharding a 40 GB database is like hiring four drivers for one auto-rickshaw. Impressive org chart, same passenger.

TINY CHALLENGE

A table takes 40 writes per second and 9,000 reads per second, and is 50 GB. Which limit, and the first fix? Answer: Read throughput. Cache, then read replicas. No sharding.

MEMORY HOOK: COPY TO READ, SPLIT TO WRITE

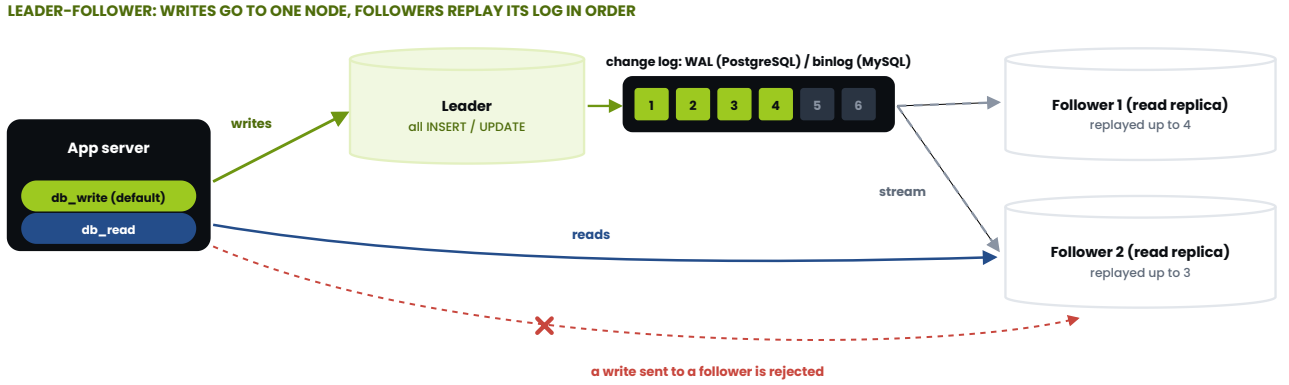
Copy it when reads or availability hurt. **Split** it when writes or storage hurt.

CONCEPT 2 · LEADER-FOLLOWER REPLICATION AND READ REPLICAS

One pen, many photocopies

The problem it solves: spread reads over several machines and keep a warm spare for when the leader dies.

- All writes go to the **leader**.
- The leader records every change in its **log** (WAL in PostgreSQL, binary log in MySQL) [S1, S2].
- Each **follower** streams that log and replays it, in the same order, so it ends up with the same data.
- Reads can go to any follower. **Writes never go to a follower.**



Async by default in PostgreSQL and MySQL [S1, S2]: followers are a little behind. Replicas scale reads and give a warm spare; they do nothing for writes.

PostgreSQL's streaming replication sends WAL records to the standby as they are generated, without waiting for a log file to fill; it is asynchronous by default, with **"a small delay between committing a transaction in the primary and the changes becoming visible in the standby"**, typically under one second if the standby keeps up [S1]. MySQL replication is also asynchronous by default: **"the source does not know whether or when a replica has retrieved and processed the transactions"** [S2].

CONCEPT 2 · HOW MANY REPLICAS FOR SHO.RT?

Replicas scale reads, never writes

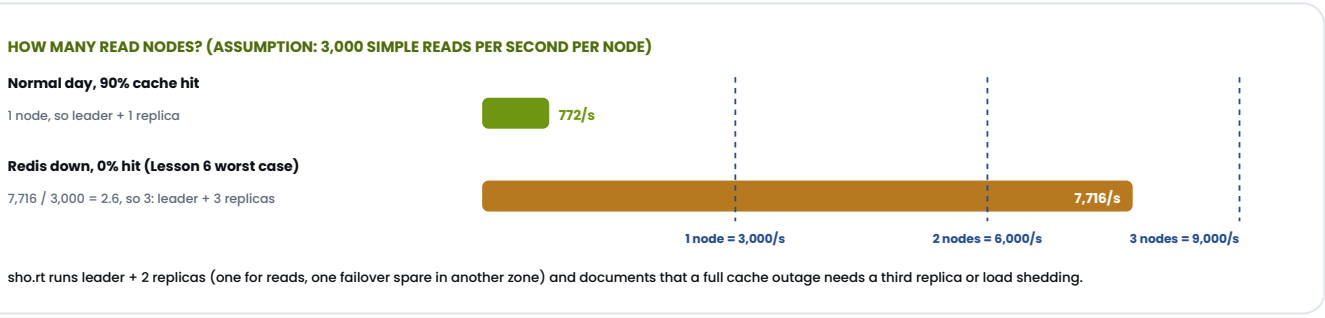
PAUSE AND PREDICT

sho.rt adds a third read replica. Does the leader now do less write work?

Answer: No. Every replica still applies all 77 writes per second, and the leader still does every write.

Worked example. Assumption: one node comfortably serves **3,000 simple reads per second**.

SCENARIO	READS/S AT THE DATABASE	NODES NEEDED FOR READS	WITH ONE SPARE
Normal day, 90% cache hit	772	1	leader + 1 replica
Redis cluster down, 0% hit (Lesson 6's worst case)	7,716	7,716 / 3,000 = 2.6, so 3	leader + 3 replicas



So sho.rt runs **leader + 2 replicas** normally (one for reads, one as failover spare in another availability zone) and documents that a full cache outage needs a third replica or load shedding. Notice the replicas did nothing for writes: each replica still applies all 77 writes per second.

WHAT GOES WRONG

Sending writes to a replica by mistake (a misconfigured connection string). PostgreSQL standbys reject writes; your app then fails in production at 2 am on the code path nobody tested. Keep two connection pools, `db_write` and `db_read`, and make the write pool the default.

TINY CHALLENGE

sho.rt has 2 replicas. How many writes per second does each one apply at peak? Answer: All 77. Replicas copy every write.

MEMORY HOOK: ONE PEN

Only **one pen** writes on the master copy. Everyone else reads **photocopies**.

INTERVIEW MODE: SAY IT LIKE THIS

- "All writes go to the leader, which streams its log to followers; followers replay it and serve reads."
- "Replicas scale reads and give a warm spare for failover; they do nothing for write throughput."

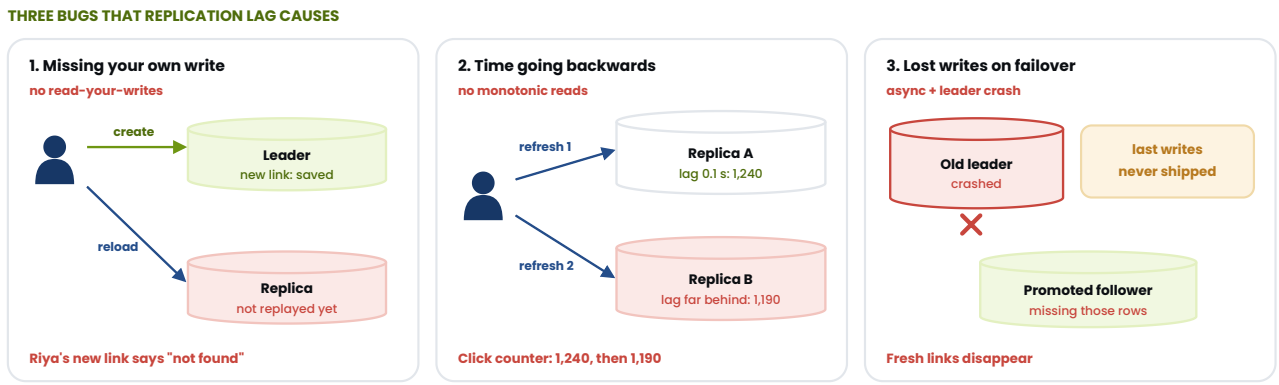
CONCEPT 3 · REPLICATION LAG AND THE THREE BUGS IT CAUSES

Slightly old data, three different bugs

Once reads come from replicas, a user can see data that is slightly old. You need to know which flows can tolerate that and which cannot.

How it works. Lag is normally small, but it grows when the replica is busy (a heavy analytics query), when the network is slow, or when the leader takes a burst of writes. Three bugs follow:

BUG	WHAT THE USER SEES	SHO.RT EXAMPLE
Missing your own write (no read-your-writes)	You create or change something, reload, and it is not there	Riya's new link says "not found" (the hook)
Time going backwards (no monotonic reads)	Two reads in a row hit replicas with different lag; the second shows older data	The click counter on the dashboard shows 1,240, then 1,190 on refresh
Lost writes on failover	The leader dies; writes it acknowledged but had not shipped are gone	Links created in the last fraction of a second before the crash disappear



PAUSE AND PREDICT

A user refreshes twice and the two reads land on replicas with different lag. What can they see?

Answer: Data going backwards in time: the second read shows an older value. That is a missing **monotonic reads** guarantee.

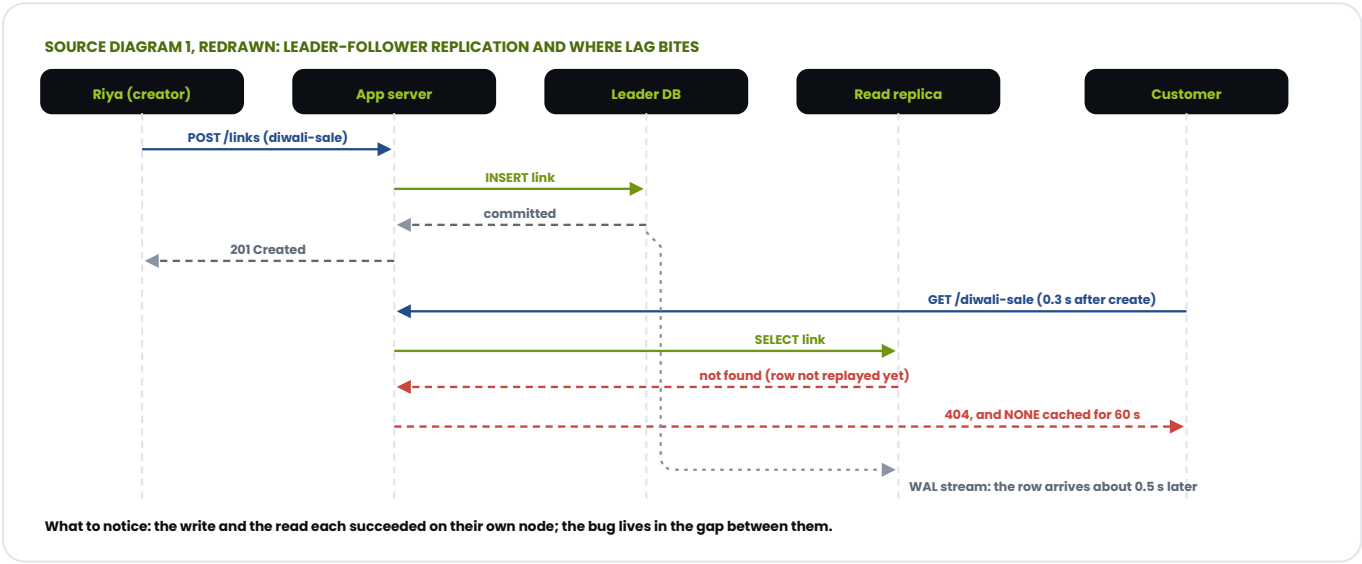
CONCEPT 3 · MEASURING LAG, AND THE HOOK IN NUMBERS

Measure it end to end

Measuring lag (real commands). On a PostgreSQL leader, `pg_stat_replication` has one row per connected standby; for an asynchronous standby, `replay_lag` **"approximates the delay before recent transactions became visible to queries"** [S11]:

```
-- run on the PostgreSQL primary
SELECT application_name, state, write_lag, flush_lag, replay_lag
FROM pg_stat_replication;
```

On MySQL 8.0.22 and later, `SHOW REPLICA STATUS` reports `Seconds_Behind_Source`. Its own manual warns that this column **"is useful only for fast networks"**, because it compares the replica's two threads, not the replica with the source [S12]. Production teams therefore often also write a timestamp row on the leader every second and read it on each replica (a **"heartbeat"**), which measures lag end to end.



Worked example (the hook, with numbers). Assumptions: replica lag at 9 pm is 0.5 s; 2% of new links are clicked within half a second (people paste into broadcast lists); peak creations are 77 per second.

- Broken first clicks: 77 x 0.02 = **about 1.5 links per second** get a "not found" on first tap.
- Each one is then negatively cached for 60 s (Lesson 6), so for each such link every click in that minute fails.
- Per peak hour: 1.5 x 3,600 = **about 5,500 links** start life broken.

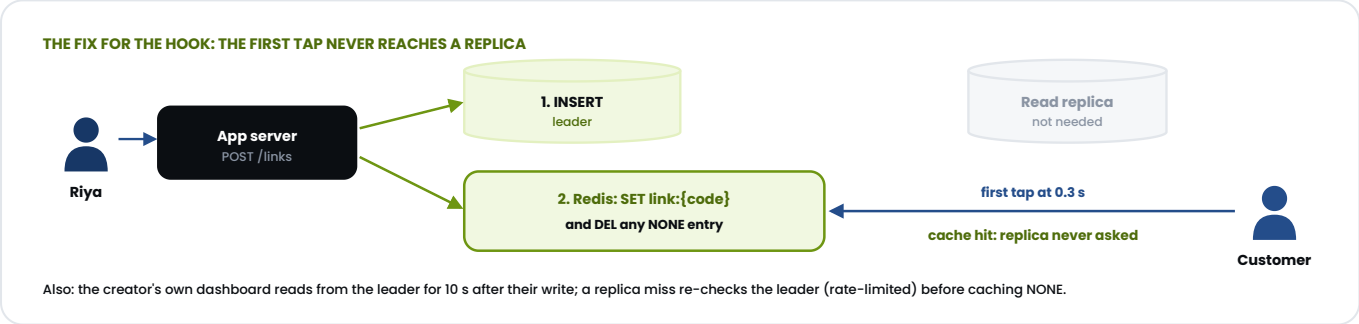
CONCEPT 3 · THE FIXES, CHEAPEST FIRST

Fix each bug with a specific technique

Fixes for "missing your own write", cheapest first:

1. **Write-through to the cache on create.** When a link is created, SET link:{code} in Redis immediately and delete any NONE entry (Lesson 6). The first redirect is a cache hit and never reaches a replica. Cost: one extra Redis call per creation (77 per second).
2. **Read from the leader for a short window after a write.** For the creator's own session (their dashboard), route reads to the leader for, say, 10 seconds after any write they make. Cost: a little extra leader load.
3. **Never cache a miss that came from a replica without checking the leader.** On a replica miss, re-check the leader before writing NONE. Cost: Lesson 6's bot probes (about 1,000 random codes per second) would now hit the leader; add a rate limit.
4. **Synchronous replication with remote_apply.** PostgreSQL can make each commit wait until synchronous standbys have replayed the transaction, "making it visible to user queries" [S1]. Cost: every write waits for a round trip and a replay.

sho.rt uses **1 + 2**, and keeps **3** behind a rate limit.



Fix for "time going backwards": pin each user's reads to one replica (for example, `replica = hash(user_id) % number_of_replicas`), so successive reads never jump to a replica that is further behind. If that replica dies, the user moves to another and may see one small jump; that is accepted. **Fix for "lost writes on failover":** see Concept 4.

> **WHAT GOES WRONG**
> **Testing on a laptop where the "replica" is the same database.** Lag is zero, every test passes, and the bug appears only in production at peak.

Just for fun. Replication lag is the friend who says "I'm 5 minutes away" from his bed. Technically in sync. Practically not.

CONCEPT 3 · SAY IT IN THE INTERVIEW

Mine, backwards, gone

TINY CHALLENGE

Lag is 0.5 s, 150 links are created per second at peak, and 2% are clicked within half a second. Broken first taps per second? Answer: 150 x 0.02 = 3 per second.

MEMORY HOOK: THE THREE LAG BUGS

Mine (my write is missing), **Backwards** (time goes backwards), **Gone** (writes lost on failover). Fixes: leader reads or cache write-through; pin to one replica; semi-sync or sync.

INTERVIEW MODE: SAY IT LIKE THIS

- "Replicas lag the leader, usually by under a second but more under load. I measure it with pg_stat_replication.replay_lag or a heartbeat row."
- "Lag causes three bugs: users miss their own writes, data goes backwards between reads, and acknowledged writes can be lost on failover."
- "I fix read-your-writes with cache write-through and brief leader reads after a user's write, and monotonic reads by pinning a user to one replica."

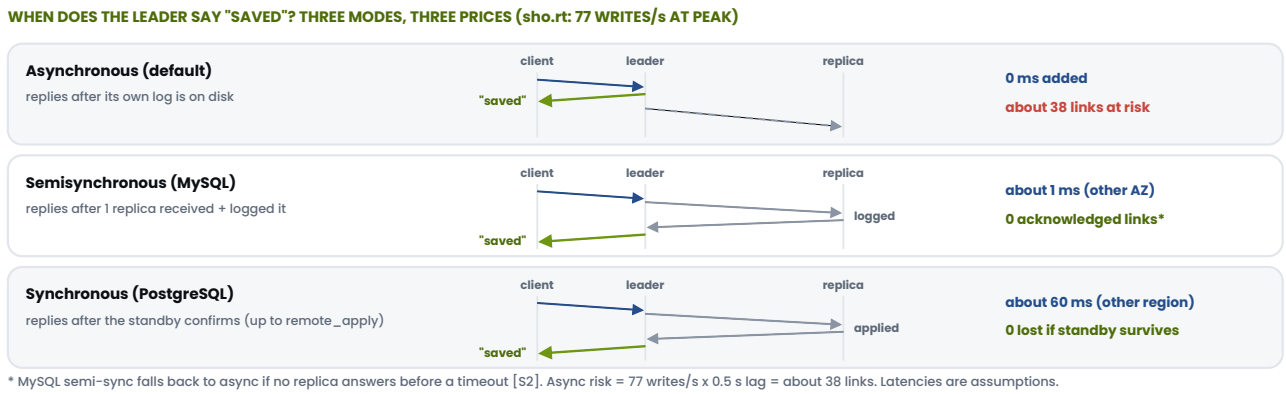
BUG	CHEAPEST FIX	HEAVIER FIX
Missing your own write	Write-through to cache on create; leader reads for 10 s after a write	Synchronous replication with remote_apply
Time going backwards	Pin each user to one replica: hash(user_id) % replicas	Read everything from the leader (wastes the replicas)
Lost writes on failover	Semi-sync or sync to one same-region replica (Concept 4)	Sync to several replicas (slow; see Concept 4's trap)

CONCEPT 4 · SYNCHRONOUS, SEMISYNCHRONOUS, ASYNCHRONOUS

Paying for durability

With asynchronous replication, a leader crash can lose writes it already confirmed to users. You need to choose how many confirmed writes you are willing to lose, and what latency you will pay to lose none.

MODE	LEADER REPLIES TO THE CLIENT WHEN...	ON LEADER CRASH	WRITE LATENCY COST
Asynchronous (default)	its own log is on disk	Writes not yet shipped are lost; "the amount of data loss is proportional to the replication delay at the time of failover" [S1]	None
Semisynchronous (MySQL)	at least one replica has received and logged the change (count configurable) [S2]	Every acknowledged write exists in at least two places [S2]	At least one network round trip per commit [S2]
Synchronous (PostgreSQL synchronous_standby_names set)	the listed standbys confirm, at the level set by synchronous_commit (remote_write, on, remote_apply) [S1]	No acknowledged write is lost if a synchronous standby survives	Round trip plus the standby's write, flush or replay time



* MySQL semi-sync falls back to async if no replica answers before a timeout [S2]. Async risk = 77 writes/s x 0.5 s lag = about 38 links. Latencies are assumptions.

PAUSE AND PREDICT

A MySQL semi-sync replica stops answering for 30 seconds. Are writes blocked forever?

Answer: No. After a timeout with no acknowledgement, MySQL falls back to asynchronous, and returns to semi-sync when a replica catches up [S2].

CONCEPT 4 · TWO DETAILS, AND SHO.RT'S CHOICE

One synchronous replica, the rest asynchronous

Two details interviewers like:

- MySQL semisynchronous replication **falls back to asynchronous** if no replica acknowledges before a timeout, and returns to semisynchronous when a replica catches up [S2]. So "we use semi-sync" does not mean "we can never lose a write" during a long network problem.
- PostgreSQL can require acknowledgements from **any two of three** listed standbys (the ANY method), a quorum-style setting [S1]. With only one synchronous standby, if it crashes, commits can stall until it is replaced [S1].

Worked example (sho.rt). Assumptions: 77 writes per second at peak; asynchronous lag 0.5 s at peak; round trip to a standby in another availability zone of the same region 1 ms; to another region (Mumbai to Singapore, say) 60 ms.

CHOICE	WRITES AT RISK ON A LEADER CRASH	ADDED LATENCY PER WRITE
Async, 0.5 s lag	77 x 0.5 ≈ 38 links	0 ms
Semi-sync to a replica in another AZ	0 acknowledged links (unless it fell back to async)	about 1 ms
Sync to a replica in another region	0	about 60 ms on every creation

For sho.rt, losing 38 freshly created links once in a rare crash is bad but survivable, and 1 ms is cheap: **choose semi-synchronous (or synchronous) to one same-region replica, asynchronous to the rest.** A payments ledger would choose synchronous to at least one replica and accept the latency.

WHAT GOES WRONG

Making every replica synchronous. Each commit now waits for the slowest one, and any one replica failing can block writes.

Just for fun. Waiting for every replica to confirm is the group-project WhatsApp poll: the decision is final only when the one guy who never opens WhatsApp votes.

TINY CHALLENGE

Async lag grows to 1.5 s at 77 writes per second. Roughly how many acknowledged links can a crash lose? Answer: 77 x 1.5 ≈ 116.

MEMORY HOOK: TRUST, RECEIPT, SIGNATURE

Async = trust me. **Semi-sync** = a receipt that it arrived. **Sync** = a signature that it was filed.

INTERVIEW MODE: SAY IT LIKE THIS

- "Async is fastest but can lose writes not yet shipped; the loss is proportional to lag at failover."
- "I choose semi-sync or sync to one same-region replica where losing acknowledged writes is unacceptable, and async to the rest."

CONCEPT 5 · MULTI-LEADER REPLICATION

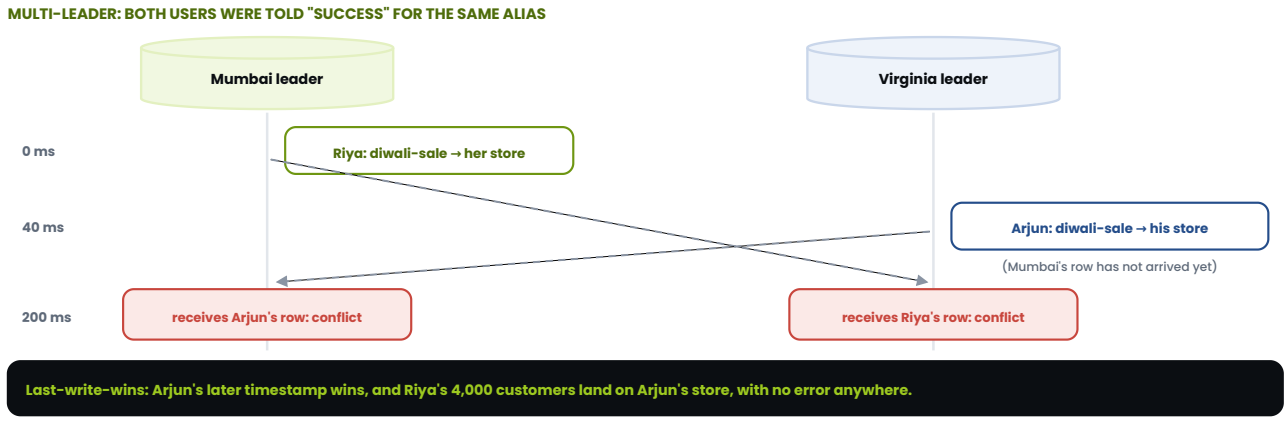
Two pens on one page start a fight

With one leader in Mumbai, a user in the United States pays a long round trip on every write, and the whole product's writes stop if Mumbai's leader region is down.

How it works. Each region has its own leader. Local writes are accepted locally and replicated to the other leaders asynchronously. Multi-master systems are typically chosen for availability and faster response, but most are only loosely consistent, and conflict resolution **"can become intractable as the number of nodes involved rises and latency increases"** [S10].

The conflict, concretely. sho.rt launches a US region. Custom aliases are unique: sho.rt/diwali-sale can belong to one user only.

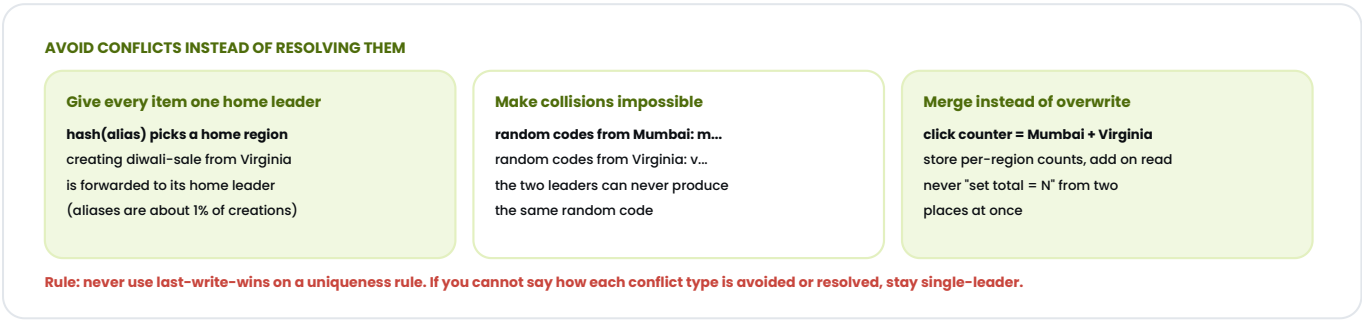
TIME	MUMBAI LEADER	VIRGINIA LEADER
0 ms	Riya creates diwali-sale → her store	
40 ms		Arjun creates diwali-sale → his store (Mumbai's write has not arrived yet)
200 ms	receives Arjun's row: **conflict**	receives Riya's row: **conflict**



Both users were told "success". Now the system must pick one. With last-write-wins, Arjun's later timestamp wins, and Riya's 4,000 customers land on Arjun's store, with no error anywhere. Dynamo describes exactly this choice: timestamp-based "last write wins", or business-specific reconciliation in the application (its shopping cart merges divergent versions) [S3].

CONCEPT 5 · AVOID CONFLICTS INSTEAD OF RESOLVING THEM

Give every item one home



- **Give every item one home leader.** Hash the alias to a home region; creating diwali-sale from Virginia is forwarded to its home leader. Custom aliases are rare (assume 1% of creations), so the extra latency hits few users.
- **Make conflicting writes impossible.** Random codes generated in Virginia start with a region character (v…), Mumbai's with m…; the two leaders can never produce the same random code.
- **Merge rather than overwrite** when the data allows it: click counters from two regions can simply be added.

Worked example (latency budget). Assumption: Virginia to Mumbai round trip about 200 ms.

DESIGN	VIRGINIA USER CREATING A RANDOM LINK	VIRGINIA USER CREATING A CUSTOM ALIAS HOMED IN MUMBAI
Single leader in Mumbai	200 ms + local work	200 ms + local work
Multi-leader, region-prefixed codes, alias homing	local only (a few ms)	200 ms (1% of creations)

WHAT GOES WRONG

Turning on multi-leader for a table with uniqueness rules, then discovering duplicates weeks later. If you cannot say how each conflict type is avoided or resolved, stay single-leader. CAP (Lesson 9) formalises the trade-off; for today, remember that multi-leader buys local writes and region independence by giving up immediate agreement.

TINY CHALLENGE

Which sho.rt data can two leaders safely merge instead of overwrite? Answer: Click counters: store per-region counts and add them.

MEMORY HOOK: TWO PENS

Two pens writing on **one page** start a fight. Give each page **one home pen**, or write on different pages.

INTERVIEW MODE: SAY IT LIKE THIS

- "Multi-leader gives local writes per region and region independence, at the cost of write conflicts."
- "I avoid conflicts by homing each item in one region or making collisions impossible; I never use last-write-wins on a unique value."

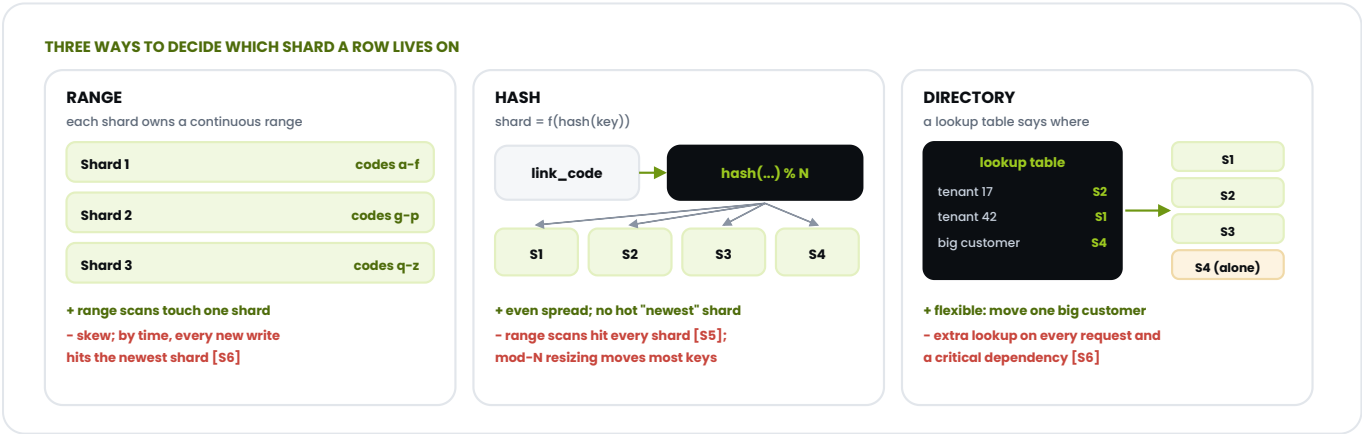
CONCEPT 6 · SHARDING STRATEGIES: RANGE, HASH, DIRECTORY

The shard key decides where every row lives

The click table needs 24 TB and 7,716 inserts per second, more than one leader should carry. You must split it so each machine holds a fraction and takes a fraction of the writes, without making the common queries painful.

How it works. The shard key decides where each row goes. Every query that includes the shard key goes to **one shard**; every query that does not must ask **all of them**. Figma states it directly: most queries need to include the shard key to be routed, and constraints like foreign keys only work along the sharding key [S5].

STRATEGY	RULE	STRENGTHS	WEAKNESSES
Range	Each shard owns a continuous range (codes a–f, dates in March)	Range scans touch one shard; easy to reason about	Uneven if values are skewed; time ranges send all new writes to the newest shard [S6]
Hash	shard = f(hash(key))	Even spread of keys; no hot "newest" shard	Range scans on the key hit every shard [S5]; changing the shard count with mod-N moves most keys
Directory	A lookup table maps key (or tenant) → shard	Maximum flexibility; can move one big customer alone	Extra lookup on every request and a critical dependency; Hello Interview calls it rarely the right interview answer for that reason [S6]



CONCEPT 6 · CHOOSING THE CLICK TABLE'S SHARD KEY

Start from the queries

The three most important queries:

- **Q1:** clicks for one link over the last 30 days (link page, very frequent).
- **Q2:** all clicks for one account's links (owner dashboard, frequent).
- **Q3:** top 10 links globally today (internal, rare).

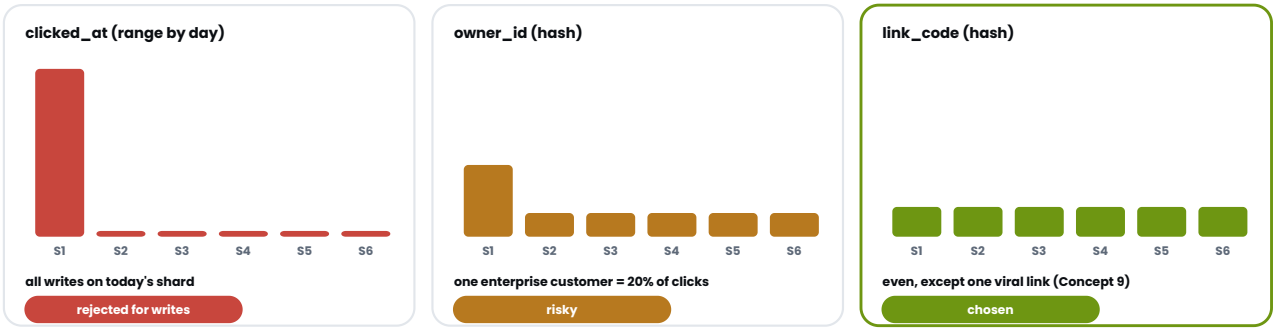
PAUSE AND PREDICT

Shard clicks by clicked_at, one shard per day. Where do today's 7,716 inserts per second go?

Answer: All of them to today's shard; every other shard sits idle.

CANDIDATE KEY	Q1	Q2	Q3	WRITE SPREAD AT 7,716/S	VERDICT
clicked_at (range by day)	1 shard	all shards	1 shard	All 7,716 writes on today's shard; others idle	**Rejected for writes** (but useful inside a shard, see below)
owner_id (hash)	1 shard	1 shard	all	Uneven: one enterprise customer can own 20% of clicks, so one shard takes about 1,540/s while others take about 480/s (assumptions)	**Risky**; Notion saw the same pattern, where one large enterprise customer generates more load than many average workspaces combined [S4]
link_code (hash)	1 shard	all shards	all, but mergeable	Even across shards, except a single viral link (Concept 9)	**Chosen**

WRITE SPREAD AT 7,716 INSERTS/s FOR EACH CANDIDATE KEY (6 SHARDS SHOWN FOR SIMPLICITY)



Decision: hash on link_code; serve the owner dashboard from a per-owner daily summary sharded by owner_id; monthly partitions inside each shard for cheap retention.

CONCEPT 6 · THE DECISION, AND COLOS

Hash across servers, range inside each

Decision: hash on `link_code`, and handle Q2 with a precomputed per-owner daily summary table sharded by `owner_id` (Concept 9). Inside each shard, partition the click table **by month** so that dropping data older than two years is a cheap DROP of an old partition rather than a giant DELETE. This mixes range partitioning within a server with hash sharding across servers, which is common.

Also shard the links table by link_code when it eventually needs sharding, so a link's metadata and its clicks sit on the same shard and can be joined locally. Figma calls such a group of tables sharing one shard key a **colo**: tables in a colo support joins and full transactions when restricted to a single shard key [S5].

WHAT GOES WRONG

Choosing the key from the data model ("every row has an id, shard on id") instead of from the queries. The first dashboard query then fans out to every shard, all day.

Just for fun. Picking a shard key without listing your queries is like choosing a hostel room by the colour of the door. Fine until you need the washroom.

TINY CHALLENGE

With link_code as the key, how many shards does Q2 (one owner's clicks) touch, and how does sho.rt avoid that? Answer: All of them. sho.rt reads a per-owner daily summary sharded by owner_id instead: 1 shard.

MEMORY HOOK: QUERIES CHOOSE THE KEY

Queries choose the key, not the columns. Then test it against the biggest customer and the most viral item.

INTERVIEW MODE: SAY IT LIKE THIS

- "I choose the shard key from the most frequent queries so they hit one shard, and check it spreads writes evenly."
- "Range keeps scans cheap but creates hot shards, especially by time; hash spreads evenly but loses range scans; a directory is flexible but adds a lookup and a critical dependency."
- "For the click table: hash on link code across machines, monthly partitions inside each, and a summary table for queries by owner."

CONCEPT 7 · LOGICAL SHARDS AND RESHARDING

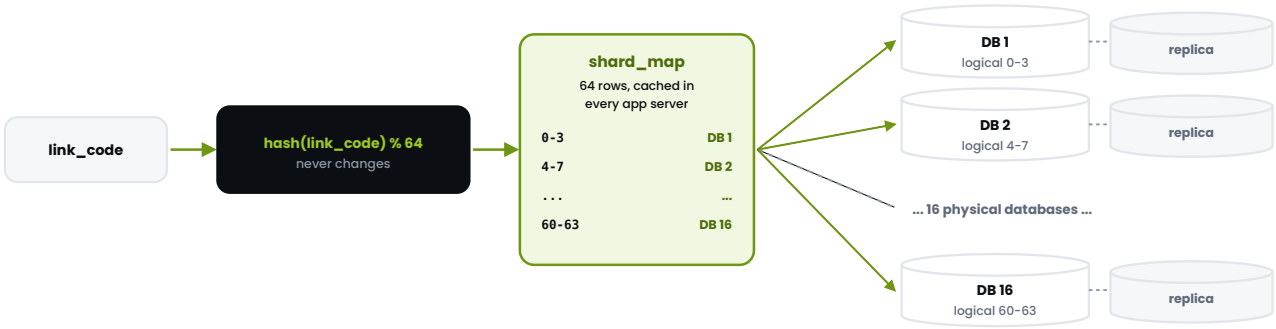
Hash once, map forever

Data grows. You will add machines. You want adding a machine to move as little data as possible, in whole, easy-to-copy units.

How it works. Do not map keys directly to machines. Map keys to a fixed, large number of **logical shards** (decided once), and map logical shards to physical machines through a small table:

```
logical_shard = hash(link_code) % 64          # never changes
physical_db   = shard_map[logical_shard]      # 64-row table, cached in every app server
```

SOURCE DIAGRAM 2, REDRAWN: LOGICAL SHARDS MAPPED TO PHYSICAL DATABASES



What to notice: the hash step never changes; only the map changes when machines are added. Resharding = move a logical shard and edit one row.

Adding a machine means moving some logical shards and updating the map. No key ever changes its logical shard, so nothing is rehashed. This is how **Notion** did it: 480 logical shards (each a Postgres schema) spread evenly across 32 physical databases, 15 per database, partitioned by workspace ID [S4]. **Figma** also separates "logical sharding" at the application layer from "physical sharding" at the Postgres layer, so it could roll out logical sharding safely before any risky physical move [S5]. **Dynamo's** later partitioning strategy follows the same idea: divide the hash space into Q equal-sized partitions, with Q much larger than the number of nodes, and assign partitions to nodes [S3].

TINY CHALLENGE

In Diagram 2, how many rows of shard_map change when you go from 16 to 32 databases? Answer: 32 rows: half of the 64 logical shards move to the 16 new databases.

CONCEPT 7 · HOW MUCH DATA MOVES?

Adding a 17th machine

Choose the logical-shard count so it divides evenly over many future machine counts. **64** splits evenly over 2, 4, 8, 16, 32 and 64 machines; Notion's **480** over 32 (and also 16, 20, 24, 30, 40, 48…) [S4].

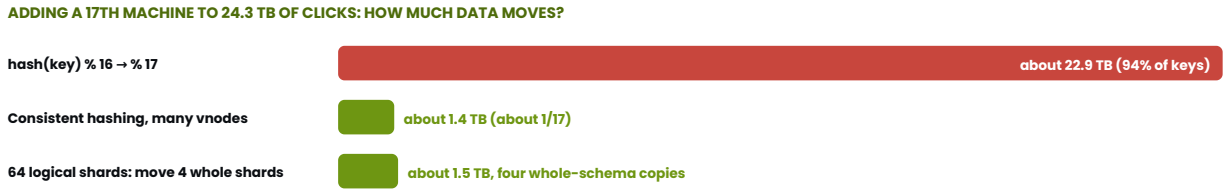
Worked example (sho.rt clicks). 24.3 TB, 64 logical shards: each logical shard is 24.3 / 64 ≈ **0.38 TB**. Start on 16 physical databases with 4 logical shards each: about **1.5 TB per machine** and 7,716 / 16 ≈ **482 inserts per second** per machine at peak.

PAUSE AND PREDICT

Going from 16 to 17 machines with hash(key) % N, what share of keys move?

Answer: Every key whose remainder changes: 1 - 1/17 ≈ **94%**.

SCHEME	WHAT MOVES	DATA MOVED
hash(key) % 16 → % 17	Every key whose remainder changes: 1 - 1/17 ≈ 94%	about 22.9 TB
Consistent hashing, many vnodes	About 1/17 of keys	about 1.4 TB
64 logical shards: move 3 or 4 whole logical shards to the new machine (64 / 17 ≈ 3.8 each)	4 x 0.38	about 1.5 TB, in four whole-schema copies



CONCEPT 7 · MOVING A SHARD WITHOUT LONG DOWNTIME

Copy, catch up, verify, flip

How a move happens online. Notion's migration from the monolith to shards is a good template [S4]:

1. **Copy** a snapshot of the logical shard to the new machine.
2. **Keep it in sync** by streaming changes (logical replication, or an audit log of writes replayed onto the new machine; Notion double-wrote through an audit log because writing directly to both databases "could quickly lead to inconsistencies") [S4].
3. **Verify:** compare row counts and checksums, or shadow-read from both and compare.
4. **Briefly pause** writes for that logical shard, let the copy catch up, **flip shard_map**, resume. Notion's final switchover was a five-minute scheduled maintenance window for the whole fleet [S4].
5. **Delete the old copy** after a safety period.



Some systems automate parts of this: MongoDB's balancer splits and migrates chunks, while Vitess supports online resharding that operators start and manage [S6].

WHAT GOES WRONG

Choosing 16 logical shards for 16 machines. The next expansion forces real resharding. Pick a logical count several times larger than any machine count you expect in the next few years. Notion's own regret was different but instructive: they would have combined the old primary key and the partition key into one column, instead of passing space_id everywhere in the application [S4].

TINY CHALLENGE

64 logical shards spread evenly on 8 machines. How many per machine? Answer: 8.

MEMORY HOOK: HASH ONCE, MAP FOREVER

The **hash** never changes. Only the small **map** does. Resharding = move a whole shard, edit one row.

CONCEPT 8 · CONSISTENT HASHING WITH VIRTUAL NODES, FOR DATA

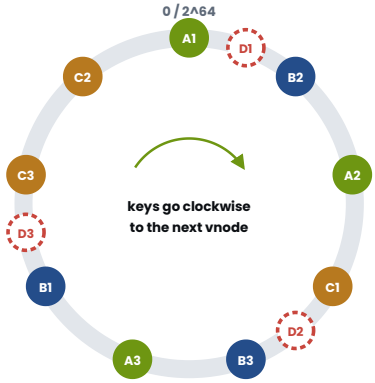
The Lesson 5 ring, now carrying data

Lesson 5 used consistent hashing to route requests to app servers, where moving a key only costs a cache miss. With databases, moving a key means copying its data. You want few moves, even load, and a way to give bigger machines more data.

How it works (recap plus the data-specific parts). Nodes and keys are hashed onto a ring; a key belongs to the first node clockwise from it [S3]. Dynamo's improvements matter for databases [S3]:

- **Virtual nodes:** each physical node owns many points on the ring, so its share is even and, when a node leaves, its load spreads across many neighbours instead of one.
- **Capacity-weighted vnodes:** "the number of virtual nodes that a node is responsible for can be decided based on its capacity", so a machine with twice the disk gets twice the vnodes.
- **Replication on the ring:** each key is stored on the next N distinct physical nodes clockwise. Because one machine owns many vnodes, the preference list skips ring positions until it has N different machines.

SOURCE DIAGRAM 3, REDRAWN: THE RING WITH VIRTUAL NODES (YOU MET THIS RING IN LESSON 5)



A, B, C = 3 machines, 3 vnodes each.
Add machine D (dashed red D1, D2, D3):
each Di takes a slice from the vnode before it, so D's ~1/4 share comes from A, B and C, not from one unlucky neighbour.

Lesson 5 used this ring to ROUTE requests:
moving a key cost one cache miss.
Here it PLACES DATA: moving a key copies its rows to another machine, so vnodes, capacity weights and replicas on N distinct machines matter much more [S3].

What to notice: with many vnodes, a joining node takes small slices from many machines, and a leaving node's data spreads across many machines.

CONCEPT 8 · AN ORIGINAL SIMULATION YOU CAN RUN

How many vnodes is enough?

Ten database nodes, 200,000 keys (MD5 hash), then an 11th node is added. Ideal: each node holds 10%, and adding a node moves 1/11 ≈ 9.1% of keys.

```
import hashlib, bisect

def h(s):
    return int.from_bytes(hashlib.md5(s.encode()).digest()[:8], "big")

def build_ring(nodes, vnodes):
    points = sorted((h(f"{n}#{i}"), n) for n in nodes for i in range(vnodes))
    return [p for p, _ in points], [n for _, n in points]

def owner(ring, key_hash):
    positions, names = ring
    i = bisect.bisect_right(positions, key_hash)
    return names[i % len(names)]          # wrap around the ring

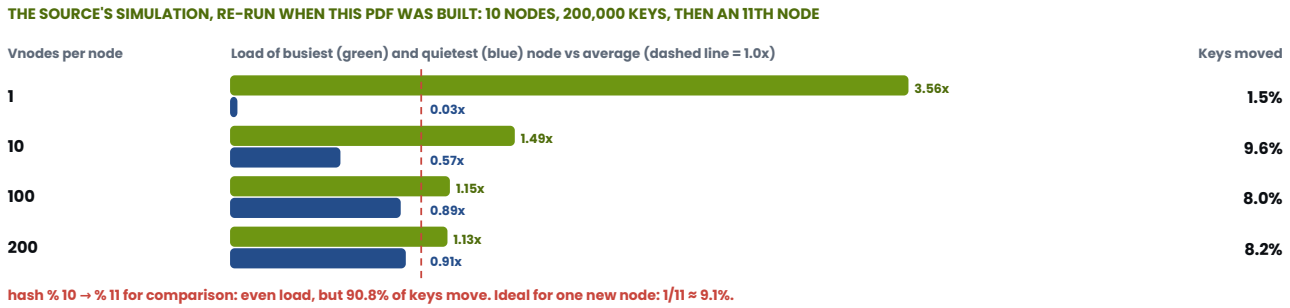
keys = [h(f"link{i}") for i in range(200_000)]
for v in (1, 10, 100, 200):
    nodes = [f"db{i}" for i in range(10)]
    ring_before = build_ring(nodes, v)
    ring_after = build_ring(nodes + ["db10"], v)
    before = [owner(ring_before, k) for k in keys]
    after = [owner(ring_after, k) for k in keys]
    counts = {n: before.count(n) for n in nodes}
    moved = sum(a != b for a, b in zip(before, after)) / len(keys)
    print(v, max(counts.values()) / 20_000, min(counts.values()) / 20_000, f"{moved:.1%}")
```

VNODES PER NODE	BUSIEST NODE VS AVERAGE	QUIETEST NODE VS AVERAGE	KEYS MOVED WHEN THE 11TH NODE JOINED
1	3.56x	0.03x	1.5% (the new node happened to land on a tiny arc)
10	1.49x	0.57x	9.6%
100	1.15x	0.89x	8.0%
200	1.13x	0.91x	8.2%
hash % 10 → % 11, for comparison	even	even	90.8%

In every ring run, **no key moved between two old nodes**; keys only moved to the new node. That is the property that makes consistent hashing cheap for data. (The source's output; this PDF re-ran the same code at build time and got identical numbers.)

CONCEPT 8 · BYTES, NOT PERCENTAGES

Many small slices, not one big bite



Worked example (bytes, not percentages). For sho.rt's 24.3 TB click table on 10 nodes, adding an 11th: mod-N moves about 0.91 x 24.3 ≈ **22 TB**; a ring with 200 vnodes moves about 0.082 x 24.3 ≈ **2.0 TB**.

Consistent hashing or logical shards? Both bound the movement to roughly the new node's fair share. Dynamo-style stores (Cassandra, DynamoDB-like systems) use rings with tokens. Teams sharding Postgres or MySQL themselves usually prefer fixed logical shards with a map (Notion, Figma), because a logical shard is a whole schema that existing backup and replication tools can copy [S4, S5].

WHAT GOES WRONG (THE ATLASSIAN QUESTION)

"What challenges arise when using consistent hashing to distribute songs across servers?" [S8]. Expect: **uneven arcs** without enough vnodes (the 3.56x row); **hot keys** (one viral song or link is one key, and the ring balances keys, not traffic per key); **replication overhead** and the need to skip same-machine vnodes; and **data movement** when nodes join or leave.

Just for fun. One vnode per server is how you get one server doing 36% of the work. Every college fest committee has that server. It is usually the treasurer.

TINY CHALLENGE

24.3 TB on 15 nodes with a good ring. Roughly how much data moves when a 16th node joins? Answer: About 1/16 of it: 24.3 / 16 ≈ 1.5 TB.

MEMORY HOOK: SMALL SLICES

With many vnodes a new node takes **many small slices** from everyone, not **one big bite** from one neighbour.

INTERVIEW MODE: SAY IT LIKE THIS

- "Consistent hashing places nodes and keys on a ring; adding a node moves only about 1 over N of keys, and only to the new node."
- "Virtual nodes even out the load, capacity-weighted vnodes give bigger machines more data, and replicas go on the next N distinct machines."

CONCEPT 9 · CROSS-SHARD QUERIES, JOINS, CONSTRAINTS AND TRANSACTIONS

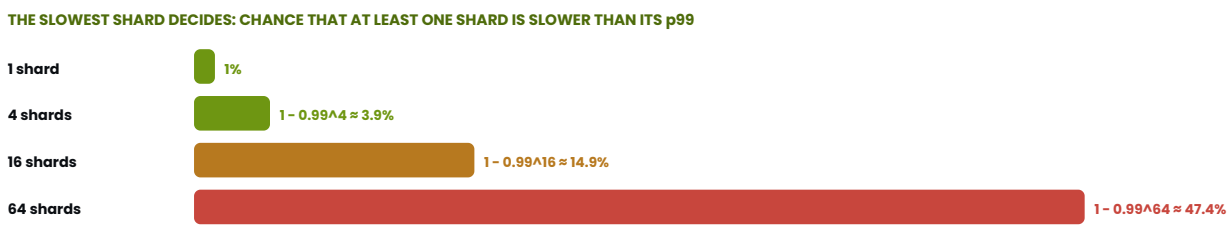
No shard key? Then what?

Some questions do not contain the shard key. You need to answer them without bringing the cluster down, and you need to know which guarantees disappear across shards. Five cases.

1. Scatter-gather. Send the query to every shard, merge the results. Figma's DBProxy parses SQL, works out which logical shards a query touches, and routes or fans it out to the right physical databases [S5].

Cost: tail latency. If each shard answers within its p99 latency 99% of the time, a query that waits for all of them is slow whenever any one is slow:

SHARDS TOUCHED	CHANCE AT LEAST ONE IS SLOWER THAN ITS P99
1	1%
4	$1 - 0.99^4 \approx 3.9\%$
16	$1 - 0.99^{16} \approx 14.9\%$
64	$1 - 0.99^{64} \approx 47.4\%$



So a scatter to 16 physical machines turns a 1-in-100 slow response into a **1-in-7** slow response. Keep scatter-gather for rare, internal or precomputed queries.

CONCEPT 9 · TOP-K ACROSS SHARDS

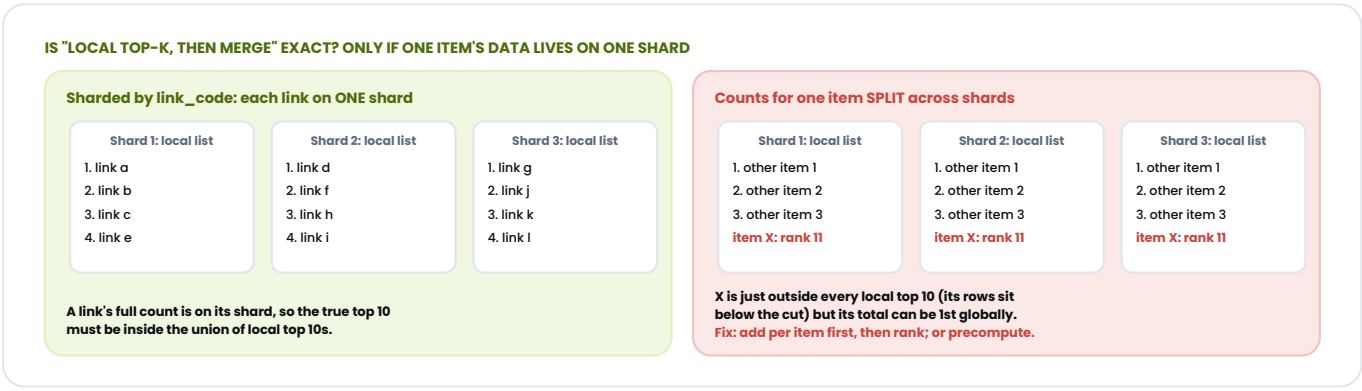
Is "merge the local top 10s" exact?

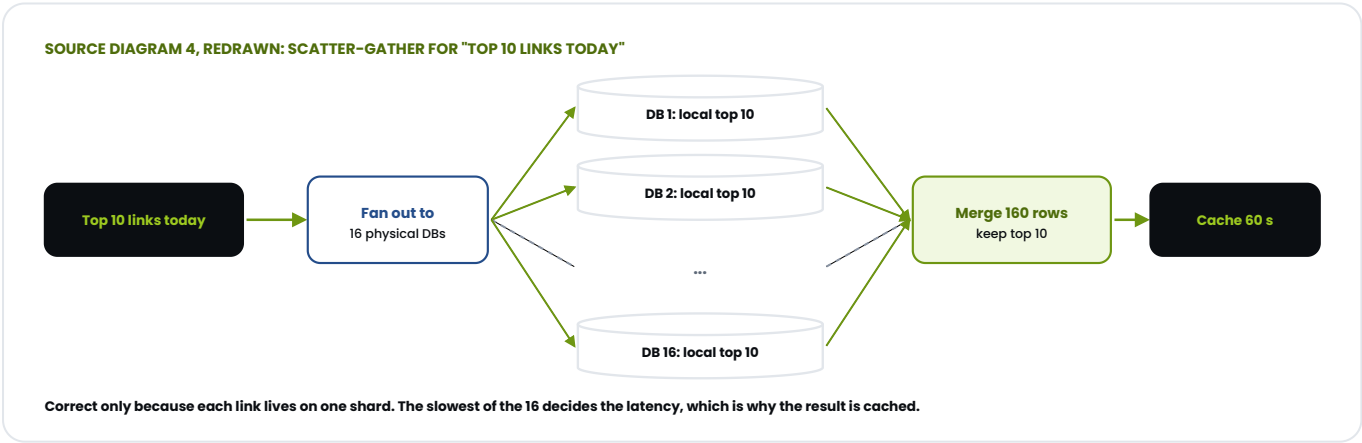
PAUSE AND PREDICT

Each of 16 machines returns its local top 10 links; you merge the 160 rows. Exact?

Answer: Yes, but only because every link lives on exactly one shard.

2. Top-k across shards (Q3: top 10 links today). Ask each shard for its local top 10 and merge. Because every link lives on exactly one shard (we shard by link_code), the true global top 10 must appear in the union of the local top-10 lists: with 16 machines, merge 160 rows, done. But if one link's counts were split across shards (see "salting" below), a link that is 11th on every shard could be 1st globally, and local top-10 merging would miss it. The fix is to aggregate per link first, or to precompute.





CONCEPT 9 · OTHER KEYS, JOINS, CONSTRAINTS, TRANSACTIONS

Index it, precompute it, or colocate it

3. Queries by another key (Q2: owner dashboard). An owner's 2,000 links are spread across all 64 logical shards. Options:

- **Scatter-gather every time** (slow, as above).
- **A global secondary index:** a separate table owner_links(owner_id, link_code) sharded by owner_id, written alongside each link. One lookup finds the codes; then fetch per shard. The index can lag the base table slightly.
- **Precompute:** a nightly or hourly job writes owner_daily_clicks(owner_id, day, clicks), sharded by owner_id. The dashboard reads one shard. sho.rt uses this, plus a "last hour" number computed live.

4. Joins and unique constraints. Joins are cheap only within one shard, so colocate tables that are joined (Figma's colos) [S5]. Unique constraints are enforced per shard: Figma notes unique indexes are only supported when they include the sharding key [S5]. For sho.rt, "alias is unique" works because the alias is the shard key. "Email is unique" for user accounts would need its own lookup table keyed by email. Auto-increment IDs also collide across shards; use globally unique IDs (a later lesson covers Snowflake-style IDs).

5. Transactions across shards. The textbook tool is **two-phase commit**: a coordinator asks all shards to prepare, then tells them to commit. It is consistent but slow and fragile; if a shard or the coordinator fails mid-way, the transaction can get stuck [S6]. Design so that each business operation touches one shard key. Figma lists atomic cross-shard transactions as future work for business-critical cases only [S5], which tells you how rarely teams need them.

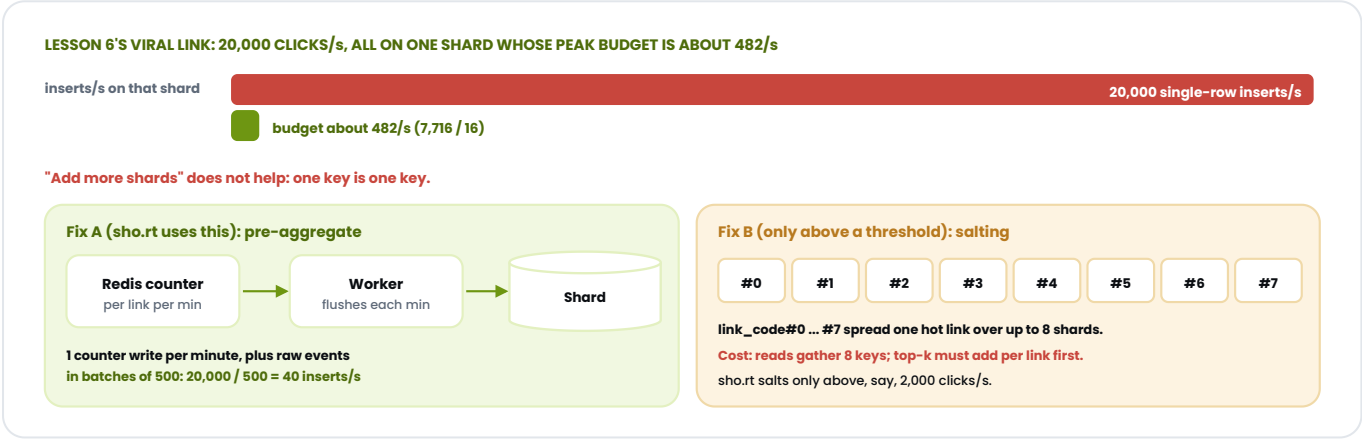
QUERY WITHOUT THE SHARD KEY	OPTION	COST
Rare, internal	Scatter-gather, cache the result	Tail latency grows with fan-out
Frequent, by another key	Global secondary index sharded by that key	Extra write; index can lag slightly
Frequent, aggregate	Precomputed summary sharded by that key	Freshness (add a live "last hour" number)

CONCEPT 9 · THE HOT-KEY PROBLEM, WITH NUMBERS

One key is one key

The hot-key problem. Lesson 6's viral link gets **20,000 clicks per second**. With hash-on-link_code, all 20,000 inserts per second land on one shard, whose budget is about **482 per second** at peak. Fixes:

- **Pre-aggregate.** Count clicks per link per minute in Redis (Lesson 6's write-back counter) and write one summary row per minute. 20,000 inserts per second become 1 write per minute for the counter, plus raw events sent asynchronously in batches.
- **Salting.** Store the hot link's raw events under link_code#0 to link_code#7 (random suffix), spreading them over up to 8 shards. Reads for that link must now gather 8 keys, and top-k merging needs the per-link aggregation step first (case 2).



sho.rt pre-aggregates for everything and salts nothing until a single link exceeds a threshold (say 2,000 clicks per second).

WHAT GOES WRONG

Adding a "small" feature, such as "search links by destination domain", that silently fans out to every shard on every keystroke. Every new query should come with the question: does it include the shard key? If not, which of the five cases is it?

TINY CHALLENGE

A dashboard fans out to 32 shards. Chance at least one is slower than its p99? Answer: $1 - 0.99^{32} \approx 27.5\%$.

MEMORY HOOK: NO SHARD KEY?

Scatter (rare), **index** (by another key), or **precompute** (aggregates). And never promise cross-shard transactions casually.

EVERYTHING SO FAR, AS ONE ANSWER

The whole lesson in interview mode

INTERVIEW MODE: SAY IT LIKE THIS

- "I first identify the limit: reads, availability, writes or storage. Reads and availability are solved with a cache, read replicas and failover; writes and storage need sharding."
- "Replicas lag, so I protect read-your-writes by writing through the cache and reading from the leader briefly after a user's write, keep monotonic reads by pinning a user to one replica, and choose semisynchronous or synchronous replication to at least one replica where losing acknowledged writes is unacceptable."
- "I choose the shard key from the main queries, prefer hash sharding into a fixed number of logical shards mapped to physical machines, and use consistent hashing or logical-shard moves so that adding a machine moves only its fair share of data."
- "Queries without the shard key become scatter-gather, secondary-index lookups or precomputed summaries, and I avoid cross-shard transactions by design."

INTERVIEW MODE: SAY IT LIKE THIS

- "I draw replication as a leader streaming its log to followers, with lag between commit and replay."
- "For sharding I draw a fixed hash into logical shards and a small map to physical databases, so resharding changes the map, not the hash. Scatter-gather fans out to every shard and is limited by the slowest one."

CONCEPT	ONE LINE	ONE NUMBER
1 · Find the limit	Copy for reads and availability; split for writes and storage	Clicks: 24.3 TB, 7,716 inserts/s
2 · Leader-follower	One writer, followers replay its log	Leader + 2 replicas for sho.rt links
3 · Lag bugs	Mine, backwards, gone	About 1.5 links/s broken at 0.5 s lag
4 · Durability modes	Async, semi-sync, sync	38 links at risk vs about 1 ms per write
5 · Multi-leader	Local writes, conflicts to avoid	Aliases: 1% of creations pay 200 ms
6 · Shard key	Chosen from the queries	link_code: even spread, 482/s per DB
7 · Logical shards	Hash once, map forever	Adding a 17th DB moves about 1.5 TB, not 22.9 TB
8 · Ring with vnodes	Many small slices	1 vnode: 3.56x busiest; 200 vnodes: 1.13x
9 · Cross-shard	Scatter, index or precompute	16 shards: 14.9% of requests hit a slow one

ONE WHITEBOARD, EVERY NUMBER

Scaling sho.rt's data layer, end to end

Requirements recap (Lessons 1 to 6). 7,716 redirects per second at peak, 3,860 on average; about 77 creations per second at peak; links rarely change; 6 billion links over five years (up to 3 TB); Redis in front with cache-aside, 60 s negative caching and a write-back click counter. **New:** per-link click analytics kept for two years; a US region launches next year.

Step 1: identify the limits per table.

TABLE	READS AT DB	WRITES	SIZE (2 TO 5 YEARS)	LIMIT HIT
links	772/s (90% hit)	77/s	up to 3 TB	Availability only
clicks	low (dashboards)	7,716/s peak	about 24 TB	**Writes and storage**

Step 2: links, replicate and do not shard. One leader, a semi-synchronous replica in a second availability zone (failover target, about 1 ms per write, zero acknowledged writes lost if it survives), and one asynchronous read replica. Redirect misses read from the read replica.

Step 3: fix the hook's bug. On creation: write to the leader, then SET link:{code} in Redis and DEL any NONE entry. Creator's dashboard reads go to the leader for 10 seconds after their write. A miss from the replica re-checks the leader before caching NONE, rate-limited to protect the leader from bot probes. Monitor replay_lag and alert above 2 seconds.

BEFORE	AFTER
about 1.5 new links per second broken on first tap at peak, each for 60 s	0 in normal operation (the first tap is a Redis hit)

Step 4: shard clicks. Shard key link_code, hashed into 64 logical shards, on 16 physical databases (4 each). Monthly partitions inside each logical shard; partitions older than 24 months are dropped.

QUANTITY	VALUE
Data per physical database	24.3 TB / 16 ≈ 1.5 TB
Peak inserts per physical database	7,716 / 16 ≈ 482/s
Each physical database	its own leader + 1 replica (32 servers in total)
Shard map	64 rows, cached in every app server, refreshed on change

STEPS 5 TO 8

Viral links, dashboards, growth, a second region

Step 5: absorb the viral link. Clicks increment a Redis counter per link per minute (Lesson 6); a worker flushes counters once a minute and writes raw events to the shards in batches of 500. A 20,000 clicks/s link produces 1 counter write per minute and **40 batch inserts per second** on its shard (20,000 / 500), instead of 20,000 single-row inserts.

Step 6: serve the cross-shard queries.

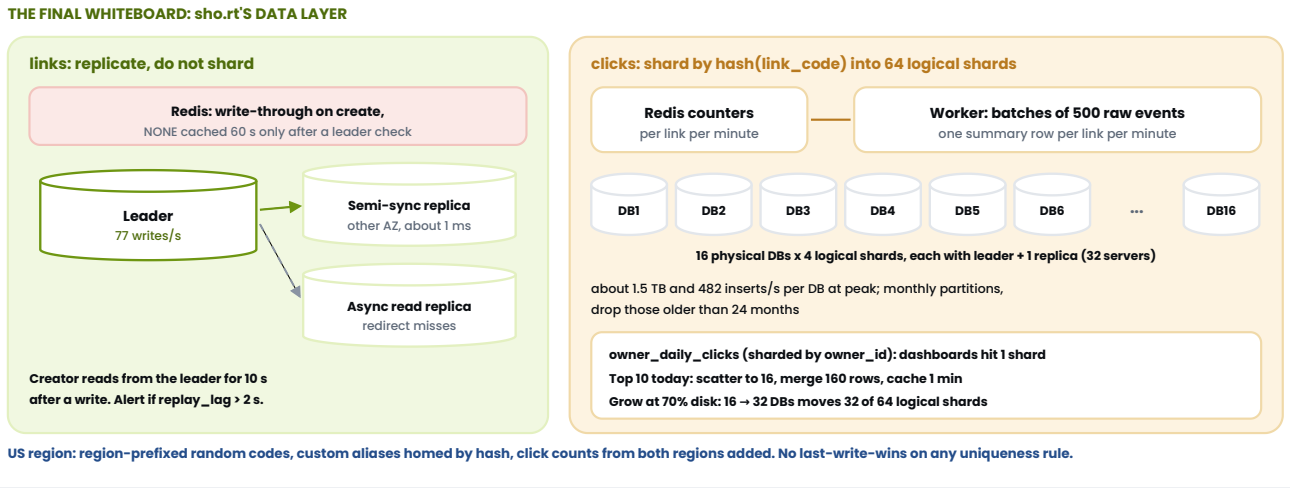
QUERY	PLAN	SHARDS TOUCHED
Clicks for one link, last 30 days	Route by link_code	1
Owner dashboard	Read owner_daily_clicks (sharded by owner_id) plus a live last-hour number	1
Top 10 links today (internal)	Scatter-gather local top 10, merge 160 rows; cached for 1 minute	16

Step 7: grow. When disks reach 70% (assumption), add machines and move whole logical shards: going from 16 to 32 machines moves 32 of 64 logical shards (half the data, about 12 TB), one shard at a time, each with snapshot, catch-up, verify, brief pause and flip.

STEP 8, AND THE FINAL WHITEBOARD

A second region, and the whole picture

Step 8: the US region. Random codes generated in Virginia carry a region prefix so they cannot collide with Mumbai's; custom aliases are homed by hash and created on their home leader; click counts from both regions are added together. No last-write-wins anywhere on a uniqueness rule.



INTERVIEW MODE: SAY IT LIKE THIS

- "For the short-link service, the links table needs replication for availability but not sharding: one leader, a semisynchronous replica in another zone and a read replica, with write-through to the cache on creation so new links never hit a lagging replica."
- "The click-analytics table needs sharding: hash on the link code into 64 logical shards on 16 databases, monthly partitions for retention, Redis pre-aggregation for viral links, a per-owner summary table for dashboards, and growth by moving whole logical shards."

THINGS THAT SOUND RIGHT AND ARE NOT

Common misconceptions

THEY SAY	THE TRUTH
"Read replicas scale writes."	Every replica applies every write. Replicas scale reads and availability only.
"Replication is instant."	PostgreSQL and MySQL replicate asynchronously by default; there is always a delay, usually small, sometimes seconds under load [S1, S2].
"Semi-sync means no data loss, ever."	MySQL semisynchronous replication falls back to asynchronous after a timeout with no acknowledgement [S2].
"Synchronous replication to all replicas is safest."	It is slowest, and one failed replica can stall commits [S1]. One synchronous replica plus asynchronous ones is the usual balance.
"Multi-leader is just leader-follower with more leaders."	It introduces write conflicts that leader-follower cannot have; you must avoid or resolve them [S10].
"Consistent hashing removes hot spots."	It balances keys, not traffic per key. One viral key is still one key on one node [S8].
"Hash sharding is always better than range sharding."	Hash loses efficient range scans on the key [S5]; range is fine when access is naturally spread, such as per-tenant ranges [S6].
"Sharding is a database feature you switch on."	For self-managed Postgres or MySQL it is mostly application design: shard keys, routing, cross-shard queries, migrations [S4, S5].
"Sharding and partitioning are the same."	People use them loosely, but partitioning often means splits inside one database server, and sharding means splits across machines [S6].

INTERVIEW MODE: SAY IT LIKE THIS

- "Replicas scale reads, not writes, and replication is asynchronous by default. Semisynchronous replication can fall back to asynchronous."
- "Consistent hashing balances keys but not hot keys. Sharding is mostly application design: choosing the key, routing queries and planning migrations."

TEAR OUT THIS PAGE

One-page cheat sheet

WHAT THEY ASK	THE ANSWER	THE TRAP
Replica vs shard?	Replica: full copy, scales reads and availability. Shard: part of the data, scales writes and storage	Saying replicas scale writes
What is replication lag?	Delay between commit on leader and visibility on a follower [S1]	Assuming it is always zero
How do you measure it?	pg_stat_replication.replay_lag [S11]; MySQL Seconds_Behind_Source (fast networks only) [S12]; heartbeat row	Trusting one metric blindly
Read-your-writes fix?	Write-through cache on create; read from leader briefly after a user's write	Reading every request from the leader
Monotonic reads fix?	Pin a user to one replica	Random replica per request
Async vs semi-sync vs sync?	Lost writes on crash vs at least one round trip vs round trip plus standby work [S1, S2]	Calling semi-sync lossless in all cases
When multi-leader?	Multi-region local writes, region independence; only with a conflict plan	Using LWW on unique data
Range vs hash vs directory?	Range: scans, but skew and hot newest shard. Hash: even, no range scans. Directory: flexible, extra lookup	Sharding time-series by time alone
Choosing a shard key?	From the top queries; high cardinality; even traffic; colocate joined tables [S5]	Choosing by "every table has an id"
Adding a shard with mod-N?	Most keys move (about 1 - 1/new count when going up by one)	Forgetting the data copy cost
Consistent hashing benefit?	Only about 1/N of keys move; vnodes even out load; capacity-weighted vnodes [S3]	One vnode per node
Logical shards?	Fixed many logical shards mapped to fewer machines; move whole shards [S4, S5]	Logical count = machine count
Cross-shard query?	Scatter-gather, secondary index, or precompute; watch tail latency	Fanning out on a hot path
Cross-shard transaction?	2PC is slow and fragile; design to one shard key [S6]	Promising ACID across shards casually
Hot key?	Pre-aggregate, cache, or salt the key	"Add more shards" (does not split one key)

INTERVIEW MODE: SAY IT LIKE THIS

- "Replicas copy data to scale reads; shards split data to scale writes and storage. Lag is measured with replay lag or a heartbeat, and read-your-writes is protected with cache write-through and brief leader reads."
- "A shard key comes from the main queries, logical shards make resharding a matter of moving whole units, and queries without the shard key are scatter-gathered, indexed separately or precomputed."

WHAT REAL TEAMS ACTUALLY DID

Where this shows up in real systems

SYSTEM	WHAT IT DID	LESSON CONCEPT
Notion (2021)	Sharded its Postgres monolith by workspace ID into 480 logical shards (Postgres schemas) on 32 physical databases; bounded tables at 500 GB and databases at 10 TB; migrated with an audit log and a five-minute maintenance window [S4]	Concepts 6 and 7
Figma (2024)	Horizontal sharding at the application layer; "colos" of tables sharing a shard key; logical sharding through Postgres views before physical splits; a DBProxy query engine that routes and fans out queries; hashed shard keys for even spread at the cost of range scans [S5]	Concepts 6, 7, 9
Amazon Dynamo (2007)	Consistent hashing with virtual nodes, capacity-based vnode counts, replicas on the next N distinct physical nodes, and later equal-sized partitions decoupled from placement [S3]	Concept 8
PostgreSQL	Asynchronous streaming replication by default; synchronous standbys with synchronous_standby_names and synchronous_commit levels up to remote_apply; lag visible in pg_stat_replication [S1, S11]	Concepts 2 to 4
MySQL	Asynchronous by default; semisynchronous mode waits for at least one replica to log the change and falls back to asynchronous after a timeout [S2]	Concept 4
Vitess, Citus, MongoDB	Sharding layers in front of MySQL and PostgreSQL, and MongoDB's balancer for chunk migration; Vitess resharding is operator-driven [S6]	Concept 7

Just for fun. Notion's announcement said "increased stability and performance". Translation: months of engineers whispering "please don't double-write wrong" at 3 am.

INTERVIEW MODE: SAY IT LIKE THIS

- "Notion and Figma sharded PostgreSQL at the application layer with logical shards mapped to physical databases and a shard key chosen from their data model."
- "Dynamo uses consistent hashing with virtual nodes. PostgreSQL and MySQL replicate asynchronously by default and offer synchronous or semisynchronous modes for durability."

NUMBERS WORTH REMEMBERING

Notion: **480** logical shards on **32** databases (15 each), **10 TB** per database, a **five-minute** switchover. Figma: queries must carry the **shard key**; unique indexes only with it. PostgreSQL lag: typically **under one second**.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 1 to 4

Two questions are Source-confirmed (PhonePe, Atlassian, both August 2025). All others are Pattern-only: commonly asked, no company source. Follow-up lines are written for this PDF from the lesson's own content.

Q1 What was the replication and sharding strategy for your project's database, and how much load did it handle? SOURCE-CONFIRMED

PhonePe SDE 2, hiring-manager round, August 2025 [S7].

Model answer. Give real numbers from your project, even small ones: "One Postgres primary and one read replica; about 50 writes and 400 reads per second at peak; no sharding because the whole database is 30 GB; if it grew, I would shard the events table by user ID into 64 logical shards." An honest small system explained well beats an imagined big one. **The GenAI project in this program gives you exactly this story.**

> **"Why no sharding?"** "Reads and availability were the limits, not writes or storage, so a cache and a replica were enough (Concept 1)."

Weak: "We used MongoDB, it scales automatically."

Strong: Real numbers, the limit you hit, and the next step you would take.

Q2 What challenges arise when using consistent hashing to distribute songs across servers? SOURCE-CONFIRMED

Atlassian SDE 2, screening round, August 2025 [S8].

Model answer. Uneven load if each server has only a few ring positions (fix: many virtual nodes, weighted by capacity); hot keys, since a viral song is one key on one server (fix: replicate or cache popular items closer to users); replicas must land on distinct physical servers; and node joins or failures still move data, so moves must be throttled and verified.

> **"How uneven, without vnodes?"** "In a 10-node simulation with one point each, the busiest node held 3.56x the average; with 200 vnodes, 1.13x."

Weak: "Consistent hashing solves load balancing."

Strong: Arcs, hot keys, replica placement, data movement, each with a fix.

Q3 What is the difference between replication and sharding? PATTERN-ONLY

Model answer. Replication copies all data to several machines to scale reads and survive failures. Sharding splits data so each machine holds a part, to scale writes and storage. Large systems do both: each shard has its own leader and replicas.

> **"Do replicas help writes?"** "No. Every replica applies every write."

Weak: "Both are ways to scale."

Strong: Copy vs split, what each scales, and that big systems do both.

Q4 What is replication lag and what problems does it cause? PATTERN-ONLY

Model answer. The delay before a follower shows a committed change; in PostgreSQL's default asynchronous mode it is usually under a second but can grow [S1]. It causes users to miss their own writes, data to appear to go backwards between reads, and lost writes on failover.

> **"How do you measure it?"** "pg_stat_replication.replay_lag on PostgreSQL, Seconds_Behind_Source on MySQL with its caveat, or a heartbeat row."

Weak: "A small delay, not a problem."

Strong: Definition, three bugs, how to measure, how to fix.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 5 to 8

Q5 Synchronous vs asynchronous replication? PATTERN-ONLY

Model answer. Asynchronous: the leader confirms after its own commit; fast, but a crash can lose writes not yet shipped [S1]. Synchronous: the leader waits for a standby; no acknowledged write lost if the standby survives, but every write pays a round trip. MySQL's semisynchronous mode waits for a replica to log the change, not apply it [S2].

> **"77 writes/s, 0.5 s lag: what can a crash lose?"** "About 38 acknowledged writes."

Weak: "Sync is safer, so always use sync."

Strong: The trade in lost writes vs milliseconds, and one sync replica plus async ones.

Q6 Leader-follower vs multi-leader? PATTERN-ONLY

Model answer. Leader-follower has one write node, so no write conflicts, but remote users pay latency and a leader failure needs failover. Multi-leader accepts writes in several places (often one per region) and must avoid or resolve conflicts [S10].

> **"Two regions create the same alias. What happens?"** "A conflict; with last-write-wins one user silently loses. Home each alias in one region instead."

Weak: "Multi-leader is faster, so it is better."

Strong: Latency and independence vs conflicts, and a plan for each conflict.

Q7 Range vs hash vs directory sharding? PATTERN-ONLY

Model answer. Range keeps ranges together and supports range scans but can create hot shards, especially by time. Hash spreads keys evenly but loses range scans on the key [S5]. Directory uses a lookup table for flexibility at the cost of an extra lookup and a critical dependency [S6].

> **"Shard a time-series by day?"** "Every new write hits today's shard. Hash across machines, partition by time inside each."

Weak: "Hash is always best."

Strong: Each one's strength, weakness, and when you would pick it.

Q8 How do you choose a shard key? PATTERN-ONLY

Model answer. From the most frequent queries: they should include the key so they hit one shard. The key should have many distinct values and spread traffic evenly, and tables that are joined should share it [S5]. Then I test it against the worst tenant or the most viral item.

> **"And the query that does not include it?"** "Secondary index, precomputed summary, or a rare scatter-gather."

Weak: "The primary key."

Strong: Queries first, cardinality, even traffic, colocation, the worst-case tenant.

FUNDAMENTALS WITH MODEL ANSWERS

Interview questions 9 to 11

Q9 What happens when you add a shard, and how do you limit data movement?

PATTERN-ONLY

Model answer. With hash % N, most keys move. Consistent hashing moves about 1/N of keys [S3]; fixed logical shards mapped to machines move whole logical shards only [S4]. The move is done online: copy, catch up, verify, brief pause, flip the map.

> **"16 to 17 machines, 24.3 TB?"** "mod-N moves about 22.9 TB; a ring about 1.4 TB; 64 logical shards about 1.5 TB in four whole copies."

Weak: "Just add the shard; the database rebalances."

Strong: Data moved under each scheme, and the online move steps.

Q10 How do you run a query that does not include the shard key?

PATTERN-ONLY

Model answer. Scatter-gather to all shards and merge, accepting tail latency; or maintain a secondary index sharded by the other key; or precompute the answer. For top-k, local top-k merging is exact only if each item lives on one shard.

> **"Fan-out to 16 shards: how often slow?"** "$1 - 0.99^{16} \approx 14.9\%$ of requests hit at least one shard's p99."

Weak: "Query all shards."

Strong: Three options, tail latency, and the top-k correctness condition.

Q11 How do you handle transactions across shards?

PATTERN-ONLY

Model answer. Avoid them by keeping each business operation within one shard key. If unavoidable, two-phase commit, which is consistent but slow and can block if a participant fails [S6]; or a saga with compensating steps (a later lesson).

> **"Why do teams avoid 2PC?"** "It is slow and fragile: a failed coordinator or shard can leave a transaction stuck. Figma lists cross-shard transactions as future work only [S5]."

Weak: "Use a distributed transaction."

Strong: Design to one shard key; 2PC or a saga only when unavoidable.

INTERVIEW MODE: SAY IT LIKE THIS

- "Replication scales reads and availability, sharding scales writes and storage. Lag causes stale reads and lost writes, so I choose the replication mode per table."
- "I pick the shard key from the main queries, use logical shards or consistent hashing to bound data movement, and design around cross-shard transactions."

HOW TO THINK WHEN IT BREAKS

Scenario questions 1 to 4

All Pattern-only. Numbers are the lesson's assumptions unless cited.

S1. Users report that edits to their link's destination "don't save", but the database shows the new value. PATTERN-ONLY

1. The edit went to the leader; the confirmation page read from a replica that had not replayed it.
2. Check replay_lag at the time of the reports.
3. Fix: after an edit, read that user's links from the leader for a short window, and update or delete the Redis entry as part of the edit (Lesson 6's delete-on-write).

S2. Replica lag jumped from 0.2 s to 40 s at 2 am every night. PATTERN-ONLY

1. A nightly analytics query or a bulk backfill is running on the replica or the leader.
2. On PostgreSQL, a long query on a standby can also conflict with replay.
3. Move analytics to a dedicated replica that read traffic does not use, throttle the backfill, and alert on lag.

S3. After a failover, about 30 links created just before the crash are missing. PATTERN-ONLY

1. Asynchronous replication lost writes that had not been shipped [S1].
2. 30 is consistent with 77 writes per second and roughly 0.4 s of lag.
3. Decide whether that is acceptable; if not, make the failover target semi-synchronous or synchronous.

S4. One shard's CPU is at 95% while the others sit at 30%. PATTERN-ONLY

1. Find out whether it is **one key** (a viral link: pre-aggregate or salt it),
2. **one tenant** (a big customer: move their logical shard to its own machine, or give them a dedicated shard through a directory entry),
3. or an **uneven hash ring** (too few vnodes).

HOW TO THINK WHEN IT BREAKS

Scenario questions 5 to 7

S5. The dashboard got slow after sharding, but each shard's queries are fast. PATTERN-ONLY

1. The dashboard query lacks the shard key, so it fans out to every shard and waits for the slowest.
2. At 16 machines, about 15% of requests hit at least one shard's p99.
3. Precompute per-owner summaries or add a secondary index sharded by owner_id.

S6. Two users in different regions both own sho.rt/launch. PATTERN-ONLY

1. Multi-leader conflict on a unique value.
2. Immediate: pick an owner by a documented rule, notify the other, and redirect them to a new alias.
3. Permanent: home every custom alias in one region so the uniqueness check happens in one place.

S7. Your interviewer says, "Just add more shards," about a viral item. PATTERN-ONLY

1. More shards do not split one key.
2. The fix must touch the key itself: cache it, pre-aggregate its writes, or salt it,
3. and then handle the extra read-side merge.

Just for fun. "Just add more shards" is the system design version of "just study harder". Correct in general. Useless for the one question you are stuck on.

INTERVIEW MODE: SAY IT LIKE THIS

- "For stale-read complaints I check replica lag and route the user's reads to the leader briefly after their writes."
- "For one hot shard I identify whether it is a hot key, a large tenant or ring imbalance, and fix each differently. For slow fan-out queries I precompute or add a secondary index on the other key."

SYMPTOM	FIRST SUSPECT	CONCEPT
"My edit didn't save", but the DB has it	Read from a lagging replica	3
Lag spikes at the same time every night	Analytics or backfill load	3
Recent writes missing after failover	Async replication	4
One shard hot, others idle	Hot key, big tenant, or too few vnodes	6, 8, 9
Fast shards, slow dashboard	Fan-out without the shard key	9
Two owners of one alias	Multi-leader conflict	5
"Add shards" does not help	One hot key	9

SOLVE THESE BEFORE YOU LOOK

Practice problems 1 to 3

Eight original problems, about 45 minutes. Numbers marked "assumption" are given to you. Every calculation was checked by running it when this PDF was built.

P1 How many replicas? EASY

A college results portal sees 12,000 reads per second at peak on result day after its cache, and 20 writes per second. Assumption: one database node serves 4,000 reads per second. The leader should take no reads, so it is free for writes and failover. How many replicas do you need with one spare? What do replicas do for the 20 writes per second?

> **Hint.** Divide, round up, add one.

SOLUTION

12,000 / 4,000 = 3 replicas for reads, plus 1 spare = **4 replicas** (5 nodes including the leader). Replicas do nothing for writes: each of the 4 replicas also applies all 20 writes per second.

P2 What does a crash cost? EASY

A wallet-top-up service writes 600 transactions per second at peak. Replication is asynchronous and lag at peak is 0.8 s. (a) Roughly how many acknowledged transactions can a leader crash lose? (b) Name a configuration change that brings this to zero for acknowledged writes, and its cost.

> **Hint.** Writes per second x seconds of lag.

SOLUTION

(a) 600 x 0.8 = **about 480 transactions**. (b) Semisynchronous replication to at least one replica (MySQL), or a synchronous standby (PostgreSQL synchronous_standby_names). Cost: at least one network round trip per commit, and with MySQL semi-sync, a fallback to async after a timeout, which must be monitored.

P3 How much data moves? MEDIUM

A 48 TB table is hash-sharded over 12 machines. You add 4 machines (16 total). How much data moves with (a) hash % 12 → hash % 16, (b) consistent hashing with many vnodes, (c) 192 logical shards spread evenly?

> **Hint.** For (a), count how many of the numbers 0 to 47 have the same remainder mod 12 and mod 16 (48 is the least common multiple).

SOLUTION

(a) A key stays only if k % 12 == k % 16. In each block of 48 consecutive values, that is true only for the first 12, so 12 / 48 = 25% stay and 75% move: **36 TB**. (A simulation over 480,000 keys gave exactly 0.75; this PDF's build re-ran it.) (b) The 4 new machines take their fair share, 4 / 16 = 25%: **about 12 TB**. (c) 192 / 12 = 16 logical shards per machine before; 192 / 16 = 12 after. Each old machine gives away 4 logical shards: 12 x 4 = 48 logical shards move, 48 / 192 = 25%: **12 TB, in 48 whole-shard copies**.

SOLVE THESE BEFORE YOU LOOK

Practice problem 4

P4 Choose a shard key MEDIUM

A college learning platform stores assignment submissions: (submission_id, student_id, course_id, assignment_id, submitted_at, file_url, marks). 2,000 colleges, 3 million students. Top queries: (1) a student's submissions across all courses (student app, very frequent); (2) all submissions for one assignment (teacher grading screen, frequent, 60 to 300 rows); (3) per-college monthly reports (rare). Propose a shard key, say which query becomes cross-shard and how you serve it, and name the hot-spot risk.

> **Hint.** Which query is most frequent? Which would be a disaster if it fanned out?

SOLUTION

One defensible answer: **shard by course_id** (hashed into logical shards). Query 2 is single-shard (an assignment belongs to one course), and so are course-level views. Query 1 becomes cross-shard: serve it from a secondary table student_submissions(student_id, course_id, assignment_id, submission_id) sharded by student_id, written alongside each submission. Query 3 is a nightly precomputed report. Alternative: shard by student_id (query 1 single-shard, query 2 cross-shard with up to 300 students' shards; serve it from a per-assignment index). Either is fine if you name the cross-shard query and its fix. **Hot-spot risk: deadline night.** All students of a large course submit in the last hour, so one course's shard takes the burst. With many courses per logical shard it averages out; for a 5,000-student course, watch that shard. Sharding by submitted_at would be the worst choice: every submission on deadline night hits the newest shard.

HOW TO CHECK YOUR OWN SHARD-KEY ANSWER

List the queries first, with how often each runs. Name the key. Say which query becomes cross-shard and which of the three options serves it (index, precompute, rare scatter). Then name the worst day: the biggest tenant, the deadline night, the viral item.

SOLVE THESE BEFORE YOU LOOK

Practice problems 5 and 6

P5 Is the merge correct? MEDIUM

A food-delivery app shards orders by restaurant_id over 32 shards. Product wants "top 20 restaurants by orders today". (a) Is "local top 20 from each shard, merge" exact? (b) Product then wants "top 20 dishes by orders today", and the same dish (for example, "paneer butter masala") is sold by restaurants on many shards. Is the same merge exact now?

> **Hint.** Can one restaurant's count be split across shards? Can one dish's?

SOLUTION

(a) **Exact.** Each restaurant lives on one shard, so its full count is on that shard; the global top 20 must be inside the union of local top 20s (32 x 20 = 640 rows to merge). (b) **Not exact.** A dish's count is split across shards; it could be 25th on every shard and 1st globally. Fix: each shard returns per-dish totals (or a per-dish partial sum for all dishes above a threshold), a merge step adds them per dish, then takes the top 20; or maintain a precomputed per-dish counter.

P6 Fan-out and the slowest shard EASY

Each shard answers within 40 ms 99% of the time. A dashboard query fans out to 8 shards and waits for all of them. (a) What fraction of dashboard requests take longer than 40 ms because of at least one slow shard? (b) What if it fans out to 50 shards?

> **Hint.** The chance all are fast is 0.99 raised to the number of shards.

SOLUTION

(a) $1 - 0.99^{8} \approx 1 - 0.9227$ = **about 7.7%**. (b) $1 - 0.99^{50} \approx 1 - 0.605$ = **about 39.5%**. Fan-out turns rare slowness into common slowness.

SOLVE THESE BEFORE YOU LOOK

Practice problems 7 and 8

P7 Spot and prevent the conflicts MEDIUM

A ticket-booking app runs one leader in Mumbai and one in Frankfurt (multi-leader, asynchronous). Which of these can conflict, and how would you prevent each? (a) Two users update their own profile photos. (b) Two users in different regions book seat 14C on the same show. (c) A show's "total tickets sold" counter is incremented in both regions.

> **Hint.** Does the item have one natural owner? Can concurrent changes be combined?

SOLUTION

(a) No conflict between different users. The same user editing from two regions at once is rare; last-write-wins is acceptable for a photo. (b) **Serious conflict:** both bookings succeed locally. Prevent it: give each show a home region and send all seat bookings for that show to its home leader (or use a single leader for bookings). Never resolve seats with last-write-wins. (c) Conflicting overwrites if stored as "set total = N", but harmless if stored as per-region counters that are added when read.

P8 Grow by moving logical shards MEDIUM

256 logical shards sit evenly on 8 machines. You grow to 10. (a) How many logical shards should each machine hold? (b) What is the minimum number of logical shards that must move, and what fraction of the data is that? (c) Compare with the ideal fair share for 2 new machines.

> **Hint.** 256 / 10 is not a whole number.

SOLUTION

(a) 256 / 10 = 25.6, so six machines hold 26 and four hold 25 (6 x 26 + 4 x 25 = 256). (b) The two new machines must end up with about 51 shards between them (for example 26 + 25). Nothing else needs to move if the old machines simply give some away: from 32 each down to 25 or 26. **51 logical shards move: 51 / 256 ≈ 19.9%** of the data. (c) The ideal is 2 / 10 = 20%. Logical shards get within one shard of ideal, and 256 does not divide evenly by 10, which is why counts like 240 or 480 (divisible by many numbers) are popular.

INTERVIEW MODE: SAY IT LIKE THIS

- "For any scaling question I size reads, writes and storage separately, compute data at risk as write rate times lag, compute data moved for mod-N, consistent hashing and logical shards, and check whether every frequent query includes the shard key."
- "If a merge across shards is involved, I check whether one item's data can be split across shards."

EIGHT QUESTIONS, TWO MINUTES EACH

Mini quiz

Original questions. Cover the answers, answer all eight, then check.

1. A database is at 90% CPU from SELECT queries. Writes are low. First move: shard, or add a cache and read replicas?

Answer: Cache and read replicas. The limit is reads.

2. True or false: adding a read replica reduces the leader's write load.

Answer: False. Replicas apply every write; the leader still does all writes.

3. Which PostgreSQL synchronous_commit setting makes each commit wait until the synchronous standby has replayed the transaction so queries there can see it?

Answer: remote_apply.

4. You shard a time-series table by day. What goes wrong?

Answer: Every new write hits today's shard; older shards sit idle.

5. Going from 20 to 21 shards with hash % N, about what fraction of keys move? With consistent hashing?

Answer: About 1 - 1/21 ≈ 95%; about 1/21 ≈ 5%.

6. Why do Notion and Figma separate logical shards from physical databases?

Answer: So that growth moves whole logical shards and edits a map, instead of rehashing keys; and logical sharding can be rolled out before physical moves.

7. A user sees their follower count drop from 312 to 309 on refresh and then return to 312. Which guarantee is missing, and the usual fix?

Answer: Monotonic reads; pin the user's reads to one replica.

8. One key receives 30% of all writes. Does adding shards help?

Answer: No. Pre-aggregate, cache or salt that key.

Just for fun. You can now explain why Riya's Diwali link broke. Riya cannot, and frankly she should not have to. That is your job now.

SEVEN SENTENCES TO KEEP

Recap and the 60-second answer

1. Find the limit first: reads and availability call for caches, replicas and failover; writes and storage call for sharding.
2. Leader-follower: writes to the leader, followers replay its log. PostgreSQL and MySQL replicate asynchronously by default, so followers lag.
3. Lag causes three bugs: missing your own write, time going backwards, and lost writes on failover. Fix them with cache write-through and brief leader reads, replica pinning, and semi-sync or sync replication to at least one replica.
4. Multi-leader buys local writes and region independence, and costs conflicts. Avoid conflicts by giving items a home leader or making collisions impossible; never use last-write-wins on unique values.
5. Range, hash and directory sharding trade range scans, even spread and flexibility. Choose the shard key from the main queries and test it against the biggest tenant and the most viral item.
6. Fixed logical shards mapped to physical machines, or consistent hashing with many vnodes, keep data movement to roughly the new machine's fair share. Move data online: copy, catch up, verify, pause briefly, flip.
7. Queries without the shard key fan out; tail latency grows with fan-out. Use secondary indexes and precomputed summaries, colocate joined tables, and design around cross-shard transactions.

SAY THIS IN AN INTERVIEW: "HOW WOULD YOU SCALE THIS DATABASE?"

"I scale reads with caches and replicas, protect users from replication lag with targeted leader reads and cache write-through, choose durability per table with synchronous or semisynchronous replication, and scale writes and storage with hash sharding into logical shards that move as whole units. Queries without the shard key are scatter-gathered, indexed separately or precomputed."

Next lesson: Lesson 8 asks which kind of database each table should live in (relational, key-value, document, wide-column, graph, search), using the access patterns you just learned to list. Lesson 9 then states CAP properly and explains consistency models and quorums.

CHECK OUR WORK

Sources

Every [S#] is a page opened in full. Every calculation in this lesson was recomputed in code when this PDF was built.

ID	PAGE	WHAT IT SUPPORTED	URL
S1	PostgreSQL 18 documentation: 26.2 Log-Shipping Standby Servers (streaming and synchronous replication)	Async by default with a small delay, typically under one second; loss proportional to delay at failover; synchronous_standby_names, synchronous_commit levels incl. remote_apply; ANY quorum; commits can stall if a sync standby crashes	postgresql.org/docs/current/warm-standby.html
S2	MySQL 8.0 Reference Manual: 19.4.10 Semisynchronous Replication	Async by default; semi-sync waits for at least one replica to receive and log events; falls back to async on timeout; at least one TCP round trip	dev.mysql.com/doc/refman/8.0/en/replication-semisync.html
S3	DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007	Ring; virtual nodes; capacity-based vnodes; preference list on distinct nodes; equal-sized partitions; LWW vs business reconciliation	allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
S4	Notion, "Herding elephants: Lessons learned from sharding Postgres at Notion" (6 Oct 2021)	Workspace ID partition key; enterprise load; 500 GB / 10 TB bounds; 480 logical shards on 32 DBs; audit-log double writes; five-minute maintenance; composite-key lesson	notion.com/blog/sharding-postgres-at-notion
S5	Figma, "How Figma's databases team lived to tell the scale" (29 Apr 2024)	Colos; logical vs physical sharding; DBProxy routing and fan-out; shard key in queries; unique indexes and foreign keys tied to it; hashed keys; cross-shard transactions as future work	figma.com/blog/how-figmas-databases-team-lived-to-tell-the-scale/
S6	Hello Interview, "Sharding" (System Design in a Hurry, 22 Oct 2025)	Partitioning vs sharding; range, hash, directory; time hot spots; directory rarely the answer; 2PC slow and fragile; MongoDB balancer, Vitess resharding	hellointerview.com/learn/system-design/core-concepts/sharding
S7	Roundz newsletter, "Interview Experience 174: PhonePe SDE 2" (30 Aug 2025)	Hiring-manager question on the replication and sharding strategy of the project's databases and the load handled	roundz.substack.com/p/interview-experience-174-phonepe-sde2
S8	Medium, Priya Goyal, "Atlassian SDE-2 Interview: My Experience & Takeaways, Part 1" (30 Aug 2025)	Screening question: challenges of consistent hashing for songs across servers	medium.com/@priyabansal0919/atlassian-sde-2-interview-my-experience-takeaways-part-1-fb83929d0066
S9	DesignGurus, "System Design Interviews Changed in 2026" (4 May 2026)	Compilation listing sharding, consistent hashing, hot keys and replication among the most frequent topics	designgurus.substack.com/p/system-design-interviews-changed
S10	Wikipedia: Multi-master replication	Definition; availability and distributed access; loose consistency and conflict-resolution difficulty	en.wikipedia.org/wiki/Multi-master_replication
S11	PostgreSQL 18 documentation: 27.2 The Cumulative Statistics System (pg_stat_replication)	write_lag, flush_lag, replay_lag; replay_lag approximates visibility delay for async standbys	postgresql.org/docs/current/monitoring-stats.html
S12	MySQL 8.0 Reference Manual: SHOW REPLICA STATUS	From 8.0.22; Seconds_Behind_Source and the fast-networks-only warning	dev.mysql.com/doc/refman/8.0/en/show-replica-status.html

DID WE COVER THE CHECKLIST?

Coverage map and honest gaps

CHECKLIST ITEM	COVERED WHERE
Read replicas and replication lag	Concepts 2 and 3, Diagram 1, worked example Steps 2 and 3, Q4, scenarios S1 to S3, Problems 1 and 2
Leader-follower vs multi-leader	Concepts 2, 4 and 5, worked example Step 8, Q5, Q6, scenario S6, Problem 7
Sharding strategies (range, hash, directory)	Concept 6, worked example Step 4, Q7, Q8, Problem 4
Resharding	Concept 7, Diagram 2, worked example Step 7, Q9, Problems 3 and 8
Consistent hashing with virtual nodes	Concept 8 (simulation), Diagram 3, Q2, Problem 3
Cross-shard queries	Concept 9, Diagram 4, worked example Step 6, Q10, Q11, scenarios S5 and S7, Problems 5 and 6

Touched lightly or left out (on purpose)

- CAP, PACELC, consistency models and quorum reads and writes: Lesson 9. Only named here.
- Leaderless replication (Dynamo-style read and write quorums, hinted handoff, read repair): Lesson 9 and the NoSQL lessons.
- Consensus and automatic leader election (Raft, Paxos) and split-brain during failover: named indirectly; a later lesson.
- Globally unique ID generation across shards (Snowflake-style IDs) and sagas for cross-shard workflows: later lessons.
- Exact lag figures depend on hardware and load; the 0.5 s, 1 ms and 200 ms values in examples are labelled assumptions.
- Company frequency: only PhonePe and Atlassian have dated reports in the window; everything else is Pattern-only.

INTERVIEW MODE: SAY IT LIKE THIS

- "This lesson covers replication, lag, multi-leader trade-offs, sharding strategies, resharding, consistent hashing and cross-shard queries. Consistency models, quorums and consensus follow in Lesson 9 and later."

SELF-CHECK

Can you say yes to all four?

- **I can find the limit**
  Reads and availability: copy. Writes and storage: split.
- **I can name and fix the lag bugs**
  Mine, backwards, gone, with a fix for each.
- **I can defend a shard key**
  From the queries, against the biggest tenant and the viral item.
- **I can size a reshard**
  mod-N vs ring vs logical shards, in TB, and the online move.

> "Riya's link was saved and still said not found." **Now you can say exactly why, and draw the fix: write-through on create, leader reads after a write, and shards that move as whole units.**

YOUR NEXT STEPS

STEP 01	STEP 02	STEP 03
Solve problems 1 to 8 on paper	Run the ring simulation yourself	Read ahead: Lesson 8, choosing a datastore